

Capacitación: Programación orientada a eventos

Pasarela de pagos GWP ("GiveMeMoney")

Documento autoguiado, por etapas, pensado para **imprimir y leer durante la semana**. Después vas a contrastar todo esto con el código real cuando tengas acceso al repo.

Antes de empezar: dos aclaraciones honestas

1. **Versiones:** la documentación interna menciona **Java 21 / Spring Boot 3.5**. Vos creías que podía ser Java 23. Como este material se armó **sin acceso al repositorio**, uso **Java 21 / Spring Boot 3.5 como referencia**. Es un dato a **confirmar** mirando los archivos de build (`pom.xml` o `build.gradle` , la propiedad `<java.version>` / `sourceCompatibility`) cuando entres al código.
2. **Nombres de código:** este documento **no inventa** nombres de clases, paquetes ni rutas de archivos. Los **exchanges, colas y eventos** sí son los reales (figuran en la doc de arquitectura). Cuando te diga "buscá la clase de configuración de RabbitMQ", es a propósito genérico: el nombre exacto lo confirmás vos en el repo.

Para qué es esta capacitación

Venís de monolitos Spring sincrónicos: un request entra, viajás por capas, llamás servicios, golpeás la base, devolvés respuesta — todo en el mismo hilo, todo "ahora". GWP es otra cosa: **microservicios que se hablan por eventos**, asíncronos y desacoplados, sobre RabbitMQ. El objetivo de la semana es que cuando abras el código no veas magia, sino patrones que ya entendés.

No te voy a explicar lo que ya sabés (qué es un `@Service` , un hilo, una transacción). Me concentro en lo nuevo: **mensajería, asincronía, hexagonal/CQRS, Outbox, consistencia eventual e infraestructura K8s**.

Mapa del documento

#	Etapas	Archivo	Lectura
	Índice (este archivo)	00-indice.md	5 min
0	Por qué eventos	etapa-0-por-que-eventos.md	15 min
1	Vocabulario de mensajería	etapa-1-vocabulario-mensajeria.md	20 min
2	RabbitMQ en este proyecto	etapa-2-rabbitmq-en-el-proyecto.md	25 min
3	Hexagonal + Clean + CQRS	etapa-3-hexagonal-clean-cqrs.md	30 min
4	Recorrido de los 4 microservicios	etapa-4-recorrido-microservicios.md	35 min
5	Patrones que tenés que entender sí o sí	etapa-5-patrones.md	35 min
6	Flujos end-to-end	etapa-6-flujos-end-to-end.md	40 min
7	Infraestructura y "verlo correr"	etapa-7-infraestructura.md	30 min
8	Consolidar	etapa-8-consolidar.md	20 min
A	Apéndice — Fundamentos de Docker y Kubernetes	apendice-A-fundamentos-docker-k8s.md	35 min
	Respuestas	99-respuestas.md	—

Total: ~5 h 25 min de lectura.

 **Sobre el Apéndice A:** la Etapa 7 te enseña a *operar* la infraestructura; el Apéndice A te enseña a *entender la maquinaria* (contenedores, labels/selectors, DNS interno, PV/PVC, Minikube) desde cero. Si nunca tocaste Docker/K8s, leelo **junto con** la Etapa 7 (o justo antes).

Cómo está armada cada etapa

Todas siguen la **misma estructura**, a propósito, para que estudies con ritmo:

-  **Objetivo** — qué te llevás, en 2-3 líneas.
-  **Teoría** — explicación didáctica. **Analogía con Java/Spring primero, código después.**
-  **En nuestro sistema** — el mapeo a GWP con los nombres reales.
-  **Qué buscar en el código (para la semana)** — qué tipo de archivo abrir cuando tengas el repo.
-  **Preguntas para verificar** — 3 a 5; las respuestas están todas juntas en 99-respuestas.md.
-  **Mini-ejercicio** — lectura y trazado mental, no escribir el sistema.

31 Ruta sugerida para la semana

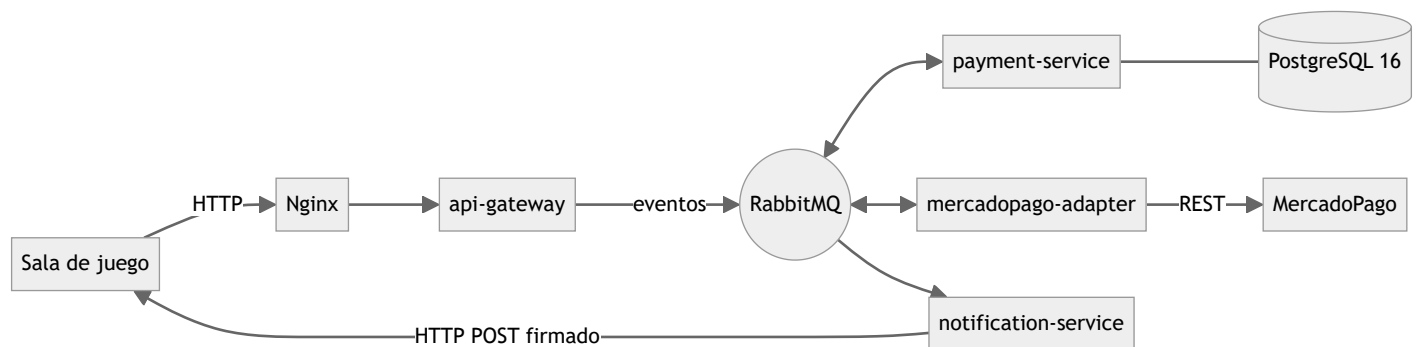
Día	Etapas	Idea
Lunes	0 + 1	El <i>porqué</i> y el vocabulario base
Martes	2 + 3	RabbitMQ concreto + organización de capas
Miércoles	4	Los 4 servicios, de una sentada
Jueves	5 + 6	Patrones + flujos completos (el corazón del sistema)
Viernes	7 + Apéndice A + 8	Infra (operar + entender la maquinaria) y cierre
Finde	repaso	Preguntas de verificación + mini-ejercicios

📖 Vista de pájaro del sistema (para tener a mano)

GWP permite a las **salas de juego** procesar pagos electrónicos vía proveedores como **MercadoPago**. Recibe solicitudes, genera órdenes QR, captura confirmaciones por webhook y notifica a los clientes — **todo asíncrono y desacoplado**.

4 microservicios:

- **api-gateway** — único punto de entrada HTTP, *stateless*, sin BD. Valida y convierte requests en eventos que publica a RabbitMQ. Responde `202 / 200` sin bloquear.
- **payment-service** — el orquestador central (*Aggregate Root*), 100% event-driven, sin HTTP. Consume 9 colas, persiste en PostgreSQL, publica a 6 exchanges. Implementa **Transactional Outbox** y tiene schedulers internos.
- **mercadopago-adapter** — adaptador de salida hacia la API de MercadoPago. *Stateless*, valida webhooks con HMAC-SHA256, reintentos con DLQ.
- **notification-service** — adaptador de salida HTTP: convierte eventos en POST firmados hacia los webhooks de cada cliente, con backoff exponencial.



El diagrama es la postal. En la **Eta**pa 6 lo recorremos mensaje por mensaje.

Siguiente: `etapa-0-por-que-eventos.md`

Etapa 0 — Por qué eventos

Objetivo de la etapa

Entender **por qué** un sistema de pagos como GWP se construye orientado a eventos en lugar de como un monolito sincrónico. Que al terminar puedas explicarle a otro las cuatro razones de fondo: **acoplamiento, escalado, tolerancia a fallos y asincronía**.

Teoría

Pensá en cómo resolverías esto con lo que ya sabés. Te piden: "cuando llega una solicitud de pago, generá una orden QR en MercadoPago, guardá todo en la base, y cuando MercadoPago confirme el pago, avisale al cliente por su webhook".

Cómo lo harías en un monolito Spring clásico

Un `@RestController` recibe el `POST`, llama a un `@Service`, que dentro de la **misma transacción y el mismo hilo**:

```
// Monolito sincrónico clásico – todo en un request
@PostMapping("/payments")
public PaymentResponse crear(@RequestBody PaymentRequest req) {
    Order order = orderService.crear(req);           // 1. lógica de negocio
    MpOrder mpOrder = mercadoPagoClient.crearOrden(order); // 2. llamada REST bloqueante
    order.setQr(mpOrder.getQr());
    orderRepository.save(order);                     // 3. persistencia
    return PaymentResponse.from(order);              // 4. respondo YA, con todo resuelto
}
```

Esto funciona, lo hiciste mil veces, y para muchos sistemas está perfecto. Pero mirá las costuras cuando el sistema crece:

- 1. Acoplamiento temporal.** El paso 2 es una llamada HTTP bloqueante a MercadoPago. Si MercadoPago tarda 4 segundos, tu request tarda 4 segundos. Si MercadoPago está caído, **tu endpoint falla**, aunque la solicitud del cliente era perfectamente válida y la podrías haber procesado más tarde. Tu disponibilidad quedó atada a la de un tercero.
- 2. Escalado en bloque.** Si te llueven webhooks de confirmación pero pocas solicitudes nuevas, no podés escalar "solo la parte de webhooks": escalás el monolito entero. Pagás CPU y memoria de todo para resolver el cuello de botella de una parte.
- 3. Tolerancia a fallos.** Si el paso 4 (notificar al cliente) falla, ¿qué hacés? ¿Rollback de todo el pago que ya está confirmado en MercadoPago? No podés: el dinero ya se movió. Mezclar "cobrar" con "avisar" en una sola transacción te deja en estados imposibles.
- 4. Asincronía forzada a mano.** Para no bloquear, terminás metiendo `@Async`, `ExecutorService`, `CompletableFuture`, colas en memoria con `BlockingQueue` ... y reinventás, mal y sin durabilidad, lo que un broker de mensajería te da hecho. Si el proceso se cae, tu `BlockingQueue` en memoria se perdió.

El cambio de mentalidad: del "llamar" al "avisar"

En event-driven dejás de **llamar** a quien sigue y pasás a **avisar que pasó algo**. El api-gateway no le dice al payment-service "hacé esto"; **publica un hecho**: "*se recibió una solicitud de pago*" (`OnPaymentRequestReceivedEvent`). Quién lo escucha, cuándo y

qué hace con eso, es problema del consumidor.

Analogía: en el monolito sos un gerente que llama por teléfono a cada empleado y espera en la línea hasta que termina. En event-driven dejás una **nota en un casillero** (la cola) y seguís con lo tuyo. El empleado la toma cuando puede. Si está de licencia, la nota lo espera. Si hay mucho trabajo, ponés tres empleados a vaciar el mismo casillero.

Esa "nota en el casillero" es **durable** (sobrevive a reinicios), **desacoplada** (el que avisa no conoce al que procesa) y permite **escalar por casillero** (agregás consumidores donde se acumula trabajo).

Lo que ganás, punto por punto

Problema del monolito	Cómo lo resuelve event-driven
Acoplamiento temporal	El gateway publica y responde <code>202</code> al instante; no espera a MercadoPago
Escalado en bloque	Escalás el servicio (o los consumidores de una cola) que está saturado
Fallos en cascada	Si un servicio se cae, los mensajes esperan en la cola; nada se pierde
Asincronía artesanal	El broker te da durabilidad, reintentos y entrega garantizada

Lo que pagás a cambio (que no te mientan)

No es gratis. Aceptás **consistencia eventual** (el estado no es coherente en todos lados *en el mismo instante*, sino *al rato*), **debugging distribuido** (un flujo cruza 4 servicios y RabbitMQ), y **complejidad operativa** (K8s, broker, observabilidad). GWP asume ese costo a propósito: en pagos, **no perder un mensaje** y **no quedar atado a la caída de un proveedor** vale más que la simplicidad de un monolito.



En nuestro sistema

GWP lleva esta idea al extremo sano:

- El **api-gateway** no tiene base de datos ni lógica de negocio pesada. Recibe el `POST /api/v1/payments`, valida, lo transforma en `OnPaymentRequestReceivedEvent`, lo publica al exchange `on_new_payment_request` y responde `202 Accepted` sin esperar nada más. Le dijo al sistema "esto pasó" y se desentendió.
- El **payment-service** consume ese evento *cuando puede*, decide qué hacer, persiste, y emite **nuevos eventos** para que el `mercadopago-adapter` cree la orden y el `notification-service` avise al cliente. Nadie llama a nadie: **todos reaccionan a hechos**.
- Si MercadoPago está caído, el evento queda en la cola del adapter y se reintenta (con DLQ). El cliente que pidió el pago **ya recibió su `202`** hace rato.

Ese `202` es la diferencia filosófica con tu monolito: **"recibí tu pedido y me hago cargo"**, en vez de **"esperá en la línea que lo resuelvo entero ahora"**.

Qué buscar en el código (para la semana)

Todavía no necesitás el repo, pero anotá para después:

- En el **api-gateway**, los controllers de entrada: fijate que los métodos **no devuelven el resultado del negocio**, sino un `202 / 200` casi inmediato. Eso confirma el desacople.
- Buscá dónde el gateway **publica** a RabbitMQ en vez de llamar a otro servicio por REST. Ese es el corazón del cambio.
- En el **payment-service**, fijate que **no hay controllers HTTP**: toda su entrada son métodos que reaccionan a mensajes. Un servicio que "no tiene puerta HTTP" es algo que en tu mundo monolítico no existía.

Preguntas para verificar que entendiste

1. En el monolito, ¿por qué la disponibilidad de tu endpoint de pagos queda atada a la de MercadoPago? ¿Cómo rompe GWP esa atadura?
2. ¿Qué significa que el api-gateway responda `202 Accepted` en lugar de `200 OK` con el pago ya resuelto?
3. Nombrá los cuatro problemas del monolito sincrónico que el enfoque event-driven ataca.
4. ¿Cuál es el principal **costo** que aceptás al irte a event-driven, y por qué en un sistema de pagos vale la pena?
5. ¿Por qué una `BlockingQueue` en memoria con un `Executor` **no** es equivalente a usar RabbitMQ?

Mini-ejercicio

Tomá el código del monolito de arriba (`crear(...)`). En una hoja, **marcá con una cruz cada punto donde una falla dejaría el sistema en un estado inconsistente o indisponible** (ej.: MercadoPago caído, la base no responde, el cliente no recibe el aviso). Después, al lado de cada cruz, escribí en una línea **quién se haría cargo de ese paso en GWP** (qué servicio) y **qué pasaría con el mensaje si ese servicio estuviera caído**. No escribas código: es trazado mental.

Siguiente: `etapa-1-vocabulario-mensajeria.md`

Etapa 1 — Vocabulario de mensajería

🎯 Objetivo de la etapa

Quedarte con el vocabulario base de mensajería **bien claro**, porque todo lo demás se apoya acá: evento, mensaje, cola, exchange, routing key, binding, productor/consumidor, ack/nack, prefetch y DLQ. Y entender los **tres tipos de exchange** (fanout, topic, direct) con los exchanges reales de GWP como ejemplo.

📖 Teoría

Evento vs mensaje

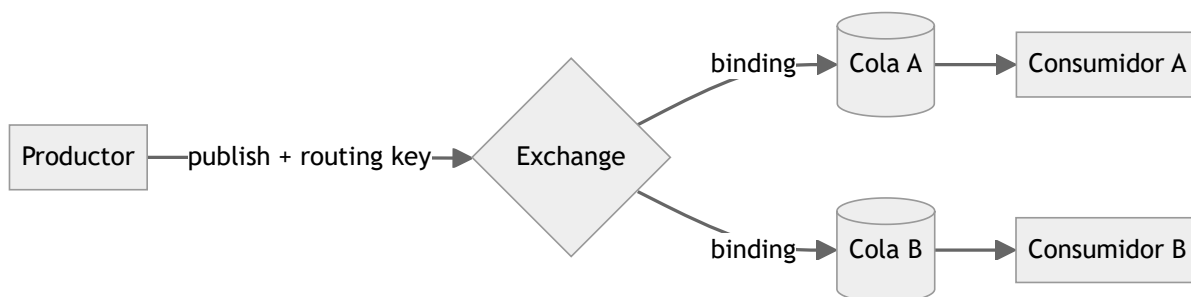
- Un **evento** es un *hecho de negocio que ya pasó*: "se recibió una solicitud de pago", "se confirmó un pago". Es conceptual, inmutable, en pasado.
- Un **mensaje** es el *sobre* que transporta ese evento por el broker: bytes (normalmente JSON) + metadatos (headers, routing key). El evento es el contenido; el mensaje es el envío.

En tu mundo: el **evento** es el DTO/ record con los datos del hecho; el **mensaje** es ese DTO ya serializado y metido en un sobre de RabbitMQ con su etiqueta de ruteo.

Las piezas de RabbitMQ

RabbitMQ implementa el protocolo **AMQP**. Tres piezas y cómo se relacionan:

- **Exchange** — la "central de correo". El productor **nunca** publica directo a una cola: publica a un *exchange*, que decide a qué cola(s) mandar el mensaje.
- **Queue (cola)** — el casillero durable donde el mensaje espera a ser consumido. Es lo único que los consumidores leen.
- **Binding** — la "regla de ruteo" que conecta un exchange con una cola. Puede llevar una condición (un patrón de routing key).



- **Routing key** — etiqueta que el productor le pone al mensaje. El exchange la compara contra los bindings para decidir el ruteo. En GWP, por ejemplo, la routing key del webhook es el **código del proveedor** (`providerCode`).
- **Productor / Consumidor** — quién publica y quién lee. En GWP el api-gateway es casi puro productor; el payment-service es las dos cosas (consume 9 colas, publica a 6 exchanges).

Analogía Spring: un `@RabbitListener` sobre una cola es, conceptualmente, como un consumidor que está dando vueltas tomando ítems de un `BlockingQueue` con un `Executor` dedicado — solo que la cola es **durable** (vive en el broker, no en tu heap), está **fuera de tu proceso**, y la entrega está **garantizada** por el protocolo.

Ack / Nack: el apretón de manos

Esto es clave y no existe en una `BlockingQueue` común. Cuando un consumidor toma un mensaje, el broker **no lo borra todavía**: lo marca como "en vuelo". El consumidor procesa y después:

- **ack** (acknowledge): "lo procesé bien, bórralo". Recién ahí desaparece de la cola.
- **nack** (negative ack): "no pude / falló". Según la config, el mensaje se **re-encola** (reintento) o se manda a la **DLQ**.

Si el consumidor **se cae antes de hacer ack**, el broker nota que la conexión murió y **vuelve a poner el mensaje en la cola** para que otro lo tome. Por eso decimos que la entrega es **at-least-once** (al menos una vez): el mensaje no se pierde, pero **puede llegar dos veces** (lo procesaste, te caíste antes del ack, y te lo redan). De ahí la importancia de la **idempotencia** — la vemos en la Etapa 5.

Prefetch: cuántos mensajes "en mano" a la vez

El **prefetch** (`prefetch count`) limita cuántos mensajes sin-ack puede tener un consumidor en vuelo simultáneamente. Si es 1, el broker no te manda el siguiente hasta que hagas ack del actual: reparto parejo, ideal para tareas pesadas. Si es alto, el consumidor "acapara" muchos y procesa más rápido, pero si se cae, más mensajes vuelven a la cola.

En tu mundo es como el **tamaño del lote** que un worker saca de la cola antes de volver a pedir. Prefetch bajo = justo a tiempo y reparto equitativo; prefetch alto = throughput pero menos balanceo.

DLQ — Dead Letter Queue

La **cola de los muertos**. Si un mensaje falla repetidas veces (o se rechaza sin re-encolar, o expira), en vez de perderse o girar para siempre, se desvía a una **DLQ**. Ahí queda para que alguien lo inspeccione: ¿por qué falló?, ¿lo reproceso a mano?, ¿es un bug? Es tu red de seguridad contra el "veneno" (un mensaje que rompe siempre y bloquearía la cola si girara infinito).

Los tres tipos de exchange

Acá está la decisión de diseño que más vas a ver. El **tipo de exchange** define *cómo* rutea:

1. `fanout` — a todos, sin mirar la routing key

Copia el mensaje a **todas** las colas atadas a él. Ignora por completo la routing key. Es "broadcast".

Cuándo: un mismo hecho le interesa a varios consumidores y no querés que el productor sepa cuántos son. Agregás un consumidor nuevo **atando otra cola** al exchange, sin tocar al productor.

En GWP: `on_new_payment_request` es **fanout, routing key ""** (vacía, justamente porque no se usa). El gateway publica "llegó una solicitud" y el sistema decide quién la consume — sin que el gateway lo sepa.

2. `topic` — por patrón de routing key

Rutea comparando la routing key contra **patrones** en los bindings, usando comodines: `*` (una palabra) y `#` (cero o más palabras), separadas por puntos. Flexible y declarativo.

Cuándo: el mismo exchange recibe mensajes de distintas "categorías" y querés que cada cola se suscriba a un subconjunto.

En GWP: `on_new_webhook_notification` es **topic**, con **routing key** = `providerCode`. Así, las notificaciones de MercadoPago (`providerCode` de MP) pueden rutearse a la cola del adapter correcto, y mañana otro proveedor rutea a otro lado — **sin tocar al gateway**, solo agregando un binding.

3. `direct` — por coincidencia exacta de routing key

Rutea a las colas cuyo binding tenga **exactamente** la misma routing key del mensaje. Sin comodines. Es el punto medio: más dirigido que fanout, más simple que topic.

Cuándo: querés mandar a una cola específica según una clave exacta y previsible.

Resumen mental:

Tipo	Mira la routing key	Reparte a...	Metáfora
<code>fanout</code>	No	todas las colas atadas	altavoz / lista de difusión
<code>topic</code>	Sí, por patrón (<code>*</code> , <code>#</code>)	las que matcheen el patrón	suscripción por temas
<code>direct</code>	Sí, exacta	las que tengan esa key exacta	sobre con destinatario puntual



En nuestro sistema

Exchanges reales que ya conocés del contexto, y qué tipo son:

- `on_new_payment_request` — `fanout`, routing key `"`. El gateway difunde "llegó una solicitud de pago". Fanout porque al gateway no le importa quién la consume.
- `on_new_webhook_notification` — `topic`, routing key = `providerCode`. Rutea la notificación según el proveedor que la mandó.
- `on_new_cancel_order_request` y `on_new_refund_order_request` — los otros dos puntos de entrada del gateway (cancelación y reembolso).
- `on_notification_attempt_completed` — exchange de **auditoría** que publica el notification-service: registra el resultado de cada intento de notificación al cliente.

Y las piezas conceptuales aparecen así:

- **Productor puro:** api-gateway.
- **Productor + consumidor:** payment-service (9 colas in, 6 exchanges out) y mercadopago-adapter (6 colas in, 5 exchanges out).
- **DLQ + reintentos:** el mercadopago-adapter los usa explícitamente para tolerar fallas de la API de MercadoPago.

📁 Qué buscar en el código (para la semana)

- La **clase de configuración de RabbitMQ** (la que declara los `Exchange`, `Queue` y `Binding` como `@Bean`). Ahí vas a ver, en código, el **tipo** de cada exchange (`FanoutExchange`, `TopicExchange`, `DirectExchange`) y los bindings con sus routing keys.
- Buscá dónde se configura la **DLQ**: típicamente argumentos como `x-dead-letter-exchange` al declarar la cola, o beans de exchange/cola con "dlq"/"dead-letter" en el nombre.
- Buscá dónde se setea el **prefetch** (suele estar en `application.yml` / `.properties` como `spring.rabbitmq.listener.simple.prefetch`, o en la config del container factory).

? Preguntas para verificar que entendiste

1. ¿Por qué un productor publica a un **exchange** y no directamente a una cola? ¿Qué gana con esa indirección?
2. `on_new_payment_request` es `fanout` con routing key `""`. ¿Por qué la routing key está vacía y qué implica para agregar un nuevo consumidor?
3. ¿Qué diferencia hay entre que el payment-service haga **ack** o **nack** de un mensaje? ¿Qué pasa si se cae **antes** de cualquiera de los dos?
4. Explicá con tus palabras por qué "at-least-once" obliga a pensar en idempotencia.
5. ¿Para qué sirve una DLQ y qué problema concreto evita (pista: mensaje "veneno")?

🔪 Mini-ejercicio

Dibujá en una hoja los tres exchanges de entrada del gateway (`on_new_payment_request`, `on_new_webhook_notification`, y uno de cancel/refund). Para cada uno, anotá: **(a)** su tipo de exchange, **(b)** qué routing key usaría el productor, y **(c)** en una frase, por qué ese tipo es el adecuado para ese caso. Para el webhook, inventá un segundo proveedor ficticio y mostrá **qué tendrías que agregar** (sin tocar el gateway) para que sus notificaciones vayan a otra cola.

Siguiente: `etapa-2-rabbitmq-en-el-proyecto.md`

Etapla 2 — RabbitMQ en este proyecto

Objetivo de la etapa

Bajar el vocabulario de la Etapa 1 a **cómo se ve en Spring**: cómo se declaran exchanges/colas/bindings, cómo un método se vuelve consumidor con `@RabbitListener`, cómo viaja el JSON, y cómo **ver todo esto funcionando** en la UI de Management de RabbitMQ.

Recordá: los snippets son **ilustrativos**. El código real de GWP puede diferir en nombres y detalles; lo confirmás cuando tengas el repo.

Teoría

Declarar la topología desde Spring

En Spring AMQP (`spring-boot-starter-amqp`), la topología — exchanges, colas, bindings — se declara como **beans**. Cuando la app arranca, un `RabbitAdmin` los crea en el broker si no existen. Esto es lindo: la infraestructura de mensajería vive **en el código**, versionada, no en clicks manuales en la UI.

```
// Ejemplo ILUSTRATIVO – el código real puede variar
@Configuration
public class RabbitConfig {

    // Un exchange fanout: difunde a todas las colas atadas
    @Bean
    FanoutExchange onNewPaymentRequestExchange() {
        return new FanoutExchange("on_new_payment_request");
    }

    // Una cola durable (sobrevive reinicios del broker)
    @Bean
    Queue paymentRequestQueue() {
        return QueueBuilder.durable("payment_request_queue").build();
    }

    // El binding: conecta la cola al exchange.
    // En fanout no hace falta routing key.
    @Bean
    Binding bindPaymentRequest(Queue paymentRequestQueue,
                               FanoutExchange onNewPaymentRequestExchange) {
        return BindingBuilder.bind(paymentRequestQueue)
            .to(onNewPaymentRequestExchange);
    }
}
```

Para un **topic exchange** el binding lleva un patrón de routing key:

```
// Ejemplo ILUSTRATIVO de topic – webhook por proveedor
@Bean TopicExchange webhookExchange() {
    return new TopicExchange("on_new_webhook_notification");
}

@Bean Binding bindMercadoPagoWebhook(Queue mpWebhookQueue, TopicExchange webhookExchange) {
    // routing key = providerCode; acá matchea exactamente el de MercadoPago
    return BindingBuilder.bind(mpWebhookQueue).to(webhookExchange).with("mercadopago");
}
```

Mentalidad: declarar un `Queue` / `Exchange` / `Binding` bean es como registrar la "cañería". No procesa nada; solo dice *cómo está conectado el sistema de correo*. El procesamiento viene aparte, con el listener.

Publicar un mensaje

El productor usa `RabbitTemplate`. Le das **exchange + routing key + payload**:

```
// Ejemplo ILUSTRATIVO – publicar el evento
rabbitTemplate.convertAndSend(
    "on_new_payment_request", // exchange
    "",                       // routing key (vacía: es fanout)
    event                     // objeto -> se serializa a JSON (ver más abajo)
);
```

`convertAndSend` aplica un **MessageConverter** para transformar tu objeto Java en bytes. El gateway hace exactamente esto: arma el `OnPaymentRequestReceivedEvent` y lo manda al exchange.

Consumir: `@RabbitListener`

Un método anotado con `@RabbitListener` se ata a una cola. Spring levanta un *listener container* (con su pool de hilos) que **escucha la cola y entrega los mensajes a tu método**. Cuando el método retorna normalmente, Spring hace **ack**; si lanza excepción, hace **nack** (y según config, re-encola o manda a DLQ).

```
// Ejemplo ILUSTRATIVO – un consumidor
@RabbitListener(queues = "payment_request_queue")
public void onPaymentRequest(OnPaymentRequestReceivedEvent event) {
    // el JSON ya se deserializó al record/clase del evento
    paymentUseCase.handle(event);
    // si retorna sin excepción -> ACK automático
    // si lanza -> NACK -> reintento o DLQ
}
```

La analogía clave: el *listener container* es tu `Executor` dedicado, la cola es tu `BlockingQueue` — pero durable y fuera del proceso —, y el `@RabbitListener` es el `Runnable` que corre por cada ítem. La diferencia enorme: el **ack** te da entrega garantizada. Con un `BlockingQueue`, si el worker explota a mitad, el ítem se perdió. Acá vuelve a la cola.

Serialización: JSON por el medio

Los servicios no comparten objetos Java en memoria: comparten **JSON** por el cable. Se configura un `MessageConverter` (típicamente `Jackson2JsonMessageConverter`) como bean, y a partir de ahí `convertAndSend` serializa a JSON al publicar y `@RabbitListener` deserializa de JSON al consumir.

Esto importa por dos razones prácticas:

1. **Contrato entre servicios:** el evento es un contrato. Si cambiás un campo de un `record` de evento en un servicio, todos los que lo consumen tienen que entender la nueva forma. Versionar eventos es un tema real.
2. **record inmutables:** GWP usa `record` para commands/events. Jackson sabe deserializar `record` desde Java 16+, así que el evento viaja como JSON y revive como `record` inmutable del otro lado.

Concurrencia y prefetch en Spring

En el `application.yml` se ajusta cuántos consumidores concurrentes hay por listener y el prefetch:

```
# Ejemplo ILUSTRATIVO
spring:
  rabbitmq:
    listener:
      simple:
        concurrency: 1      # consumidores mínimos por listener
        max-concurrency: 5  # máximo si hay backlog
        prefetch: 10       # mensajes en vuelo por consumidor
        acknowledge-mode: auto # ack/nack según retorno/excepción del método
```



En nuestro sistema

- El **api-gateway** es el productor de los 4 exchanges de entrada: publica con `RabbitTemplate` (o equivalente) hacia `on_new_payment_request`, `on_new_webhook_notification`, `on_new_cancel_order_request`, `on_new_refund_order_request`.
- El **payment-service** tiene **9 métodos consumidores** (una cola cada uno, conceptualmente) y publica a **6 exchanges**. Es el que más topología toca.
- El **mercadopago-adapter** consume 6 colas y publica 5 exchanges; además declara **DLQ** para las fallas contra MercadoPago.
- El **notification-service** consume 2 colas y publica 1 exchange de auditoría (`on_notification_attempt_completed`).
- El broker es **RabbitMQ** sobre AMQP (5672), con **UI de Management en 15672**, corriendo como `StatefulSet rabbitmq-0` en Kubernetes.

Ver todo esto en la UI de Management

La UI de Management (puerto 15672) es tu mejor amiga para entender el sistema sin leer una línea de código:

- **Exchanges:** ves los 6+ exchanges, su **tipo** (fanout/topic/direct) y sus bindings. Acá confirmás de un vistazo que `on_new_payment_request` es fanout.
- **Queues:** ves cada cola, **cuántos mensajes tiene encolados**, cuántos están "en vuelo" (unacked), y la tasa de entrada/salida. Si una cola crece y no baja → consumidor caído o lento.
- **Bindings:** ves qué cola está atada a qué exchange con qué routing key.
- Podés incluso **publicar un mensaje a mano** desde la UI para probar un consumidor.

Para llegar ahí en el entorno de GWP (lo retomamos en la Etapa 7):

```
kubect1 port-forward -n dev svc/rabbitmq-service 15672:15672
# luego abrís http://localhost:15672 en el navegador
```

📁 Qué buscar en el código (para la semana)

- La **clase de configuración de RabbitMQ** de cada servicio: los `@Bean` de `Exchange`, `Queue`, `Binding`. Es el mapa de la topología.
- El **bean del MessageConverter** (buscá `Jackson2JsonMessageConverter` o similar): confirma que el transporte es JSON.
- Los **métodos @RabbitListener**: son los puntos de entrada reales de los servicios event-driven. Contá cuántos hay en el `payment-service` (deberían ser ~9).
- Dónde se usa el `RabbitTemplate` (o un wrapper): ahí está la publicación de eventos.
- El `application.yml` / `.properties`: `concurrency`, `prefetch`, `acknowledge-mode`, y la conexión al broker.

? Preguntas para verificar que entendiste

1. ¿Qué hace un `@Bean` de tipo `Queue` / `Exchange` / `Binding` cuando arranca la app, y por qué conviene declarar la topología en código y no a mano en la UI?
2. ¿Quién hace el **ack** cuando usás `@RabbitListener` con `acknowledge-mode: auto`, y qué pasa si tu método lanza una excepción?
3. ¿Por qué los servicios intercambian **JSON** y no objetos Java directamente? ¿Qué implica eso para cambiar un campo de un evento?
4. Si entrás a la UI de Management y ves una cola con 5.000 mensajes encolados y 0 unacked, ¿qué sospechás?
5. ¿Para qué sirve `max-concurrency` y cómo se relaciona con el escalado que vimos en la Etapa 0?

🔥 Mini-ejercicio

Tomá el snippet ilustrativo del `@RabbitListener` de arriba. Anotá, paso a paso, **el ciclo de vida de un mensaje** desde que está en la cola hasta que desaparece: (1) el container lo toma, (2) se deserializa el JSON, (3) corre tu método, (4) ack/nack. En cada paso, escribí **qué pasaría si el servicio se cae justo ahí**. ¿En qué paso(s) el mensaje se reprocesaría y por qué?

Siguiente: `etapa-3-hexagonal-clean-cqrs.md`

Etapa 3 — Hexagonal + Clean + CQRS

Objetivo de la etapa

Entender **cómo está organizado por dentro** cada microservicio de GWP: las capas `domain` / `application` / `adapters`, los puertos de entrada y salida, la regla de dependencias hacia adentro, y por qué los **commands** están separados de los **events** y las **queries** (CQRS). Que veas que no es decoración: resuelve problemas concretos de testeo y de cambio.

Teoría

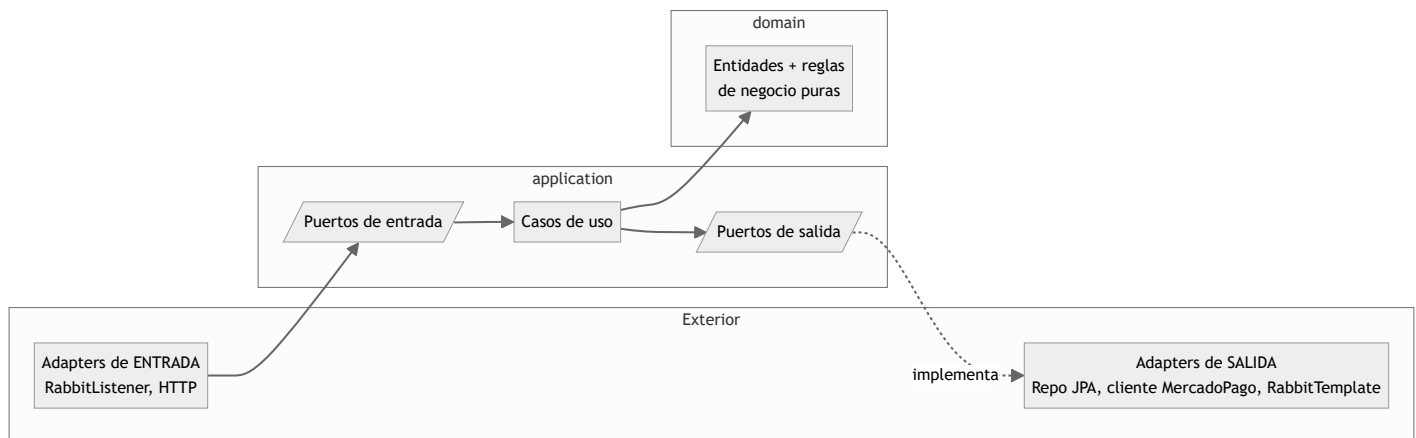
El problema que resuelve la arquitectura hexagonal

En el monolito clásico, ¿dónde vive la lógica de negocio? Muchas veces, mezclada con Spring, con JPA, con el cliente HTTP. El `@Service` importa `@Repository` (JPA), arma queries, llama clientes REST... y entonces tu **lógica de negocio depende de la base de datos, del framework y del proveedor externo**. Testearla te obliga a levantar media infraestructura, y cambiar de proveedor te obliga a tocar el corazón del negocio.

La **arquitectura hexagonal** (también "Ports & Adapters") da vuelta esa dependencia. La idea de una frase:

El negocio (dominio) no conoce el mundo exterior. El mundo exterior conoce al negocio.

El dominio define **qué necesita** mediante interfaces (los **puertos**). El mundo exterior (base, RabbitMQ, MercadoPago) provee implementaciones de esas interfaces (los **adapters**). Las dependencias **apuntan siempre hacia adentro**, hacia el dominio.



Las tres capas

- domain** — el corazón. Entidades, reglas de negocio, invariantes. **Cero dependencias** de Spring, JPA o RabbitMQ. Es Java puro. Acá vive "una orden no puede reembolsarse si no fue confirmada", "un pago expira a los X minutos", etc.
- application** — los **casos de uso** (orquestran el dominio) y la **definición de los puertos** (interfaces). Un caso de uso dice "para procesar una solicitud, valido con el dominio, persisto con el *puerto de repositorio*, y publico un evento con el *puerto*

de mensajería". No sabe **quién** implementa esos puertos.

- **adapters** — el mundo real conectado a los puertos. Dos familias:
 - **Adapters de entrada** (*driving*): empujan al sistema desde afuera. En GWP, principalmente los `@RabbitListener` (y los controllers HTTP en el gateway). Toman un mensaje/request y llaman a un caso de uso.
 - **Adapters de salida** (*driven*): el sistema los usa para hablar con el mundo. El repositorio JPA, el cliente HTTP de MercadoPago, el publicador de RabbitMQ. Implementan los puertos de salida.

La regla de oro: dependencias hacia adentro

`adapters` → `application` → `domain`. **Nunca al revés.** El `domain` no importa nada de `application`; `application` no importa nada de `adapters`. Por eso el negocio se testea sin levantar base ni broker: le pasás *mocks* de los puertos.

Analogía: el puerto de salida `PaymentRepository` (interface en `application`) es como un `interface DAO` que vos definís según lo que **el negocio necesita** ("guardame esta orden", "traeme la orden por uuid"). El adapter JPA lo implementa. El caso de uso depende de **tu interface**, no de `JpaRepository`. Cambiás de PostgreSQL a otra cosa tocando solo el adapter; el negocio ni se entera.

CQRS — separar lo que cambia de lo que pregunta

CQRS = *Command Query Responsibility Segregation*. La idea: separar las operaciones que **cambian estado** (commands) de las que **leen estado** (queries), en vez de mezclar todo en un "service" que hace ambas. En un sistema event-driven aparece una tercera categoría natural: los **eventos**.

Distinguilos así:

Tipo	Intención	Tiempo verbal	Ejemplo conceptual
Command	"hacé esto" — una orden que <i>puede fallar/rechazarse</i>	imperativo	"creá la orden en MercadoPago"
Event	"esto ya pasó" — un hecho, inmutable	pasado	<code>OnPaymentRequestReceivedEvent</code>
Query	"decime esto" — sin efectos	pregunta	"traeme el estado de la orden"

Diferencia sutil pero importante: un **command** expresa *intención* y un destinatario puede rechazarlo; un **event** es un *hecho consumado* y a quien lo escucha solo le queda reaccionar. En GWP los **commands son inmutables (record)** — una vez creados no se tocan, lo que los hace seguros para pasar entre hilos y serializar.

Por qué importa acá: el api-gateway recibe un request, lo valida y emite un **evento** (`OnPaymentRequestReceivedEvent`). El `payment-service` consume ese evento y, para que MercadoPago cree la orden, emite lo que conceptualmente es un **command** hacia el adapter. Separar estas intenciones evita el clásico "God Service" que hace todo y no se entiende.

Y todo esto compilado a binario nativo (GraalVM)

Los 4 servicios se compilan a **binario nativo con GraalVM** (*ahead-of-time*). Para vos, dos consecuencias prácticas: arranque casi instantáneo y bajo consumo de RAM (ideal para muchos pods chicos en K8s), pero **menos reflexión dinámica** disponible — por eso hexagonal/CQRS con tipos explícitos y `record` encaja tan bien, y por eso a veces ves *hints* de configuración nativa. Es un dato a tener presente, no algo que tengas que dominar esta semana.

En nuestro sistema

Los **patrones transversales en los 4 servicios** son exactamente estos: Hexagonal, Clean Architecture (dependencias hacia adentro), CQRS, commands inmutables (`record`) y compilación nativa con GraalVM.

Cómo se nota servicio por servicio:

- **payment-service** — el caso más rico. Su `domain` tiene las reglas de la orden de pago (estados, expiración, condiciones de reembolso). Sus **puertos de entrada** son los `@RabbitListener` que reciben eventos; sus **puertos de salida** son el repositorio PostgreSQL y el publicador de eventos. Es el *Aggregate Root*: el dueño de la verdad del estado del pago.
- **mercadopago-adapter** — es, él mismo, un **adapter de salida del sistema** (encapsula MercadoPago), pero internamente también está organizado en capas: puerto de entrada (consume colas), dominio finito (traducir eventos GWP ↔ API MercadoPago), puerto de salida (el cliente HTTP real).
- **notification-service** — puerto de entrada (consume 2 colas), y un puerto de salida que es el POST HTTP firmado al cliente. Sus **12 extractores de metadatos por proveedor** son un **patrón Strategy** (lo vemos en Etapa 4): una interfaz, muchas implementaciones, se elige la del proveedor en runtime.
- **api-gateway** — más finito, pero igual separa: adapter de entrada (controllers HTTP) → caso de uso (validar + transformar a evento) → puerto de salida (publicar a RabbitMQ).

Qué buscar en el código (para la semana)

- La **estructura de paquetes** de cada servicio: busca carpetas/paquetes `domain`, `application`, `adapters` (o `infrastructure / adapter`). El árbol de paquetes te cuenta la arquitectura.
- En `application`: las **interfaces de puertos** (suelen tener nombres tipo `...Port`, `...Repository`, `...Publisher`, `...UseCase`) y las clases de **casos de uso**.
- En `adapters`: las **implementaciones** de esos puertos (el repo JPA, el cliente de MercadoPago, el publicador RabbitMQ, los `@RabbitListener`).
- En `domain`: confirmá que esas clases **no importan** `org.springframework.*` ni `jakarta.persistence.*`. Si el dominio está limpio, la regla de dependencias se respeta.
- Los `record` usados como commands/events.

Preguntas para verificar que entendiste

1. ¿Por qué el `domain` no debe importar nada de Spring, JPA ni RabbitMQ? ¿Qué ganás concretamente con esa restricción?
2. ¿Qué es un **puerto de entrada** y qué es un **puerto de salida**? Dá un ejemplo de cada uno en el `payment-service`.
3. Diferenciá **command**, **event** y **query**. ¿Por qué `OnPaymentRequestReceivedEvent` es un evento y no un command?
4. ¿En qué dirección apuntan las dependencias entre `adapters`, `application` y `domain`, y por qué nunca al revés?
5. ¿Por qué tiene sentido que los commands sean `record` inmutables en un sistema que los serializa y los pasa entre hilos/servicios?

Mini-ejercicio

Tomá el caso de uso conceptual "procesar una solicitud de pago" del payment-service. En una hoja, escribí los **3-4 pasos** que haría el caso de uso (validar con el dominio, persistir, publicar evento...). Al lado de cada paso, marcá si usa el **dominio**, un **puerto de salida** o emite un **evento**. Después, identificá: si mañana cambian PostgreSQL por otra base, **¿qué capa(s) tocás y cuál NO?** Justificá en una línea.

Siguiente: `etapa-4-recorrido-microservicios.md`

Etapla 4 — Recorrido de los 4 microservicios

Objetivo de la etapa

Tener una **ficha mental clara de cada microservicio**: qué hace, qué consume, qué publica, sus casos de uso y qué buscar en su código. Al terminar deberías poder dibujar el sistema completo de memoria.

Los **números de colas/exchanges** y los **nombres de exchanges/eventos** son los reales. Los nombres de **clases/archivos** los confirmás en el repo.

Teoría (cómo leer una ficha de servicio)

Para cada servicio mirá siempre cuatro cosas, en este orden:

1. **Responsabilidad** — ¿de qué es dueño? ¿Tiene estado (BD) o es *stateless*?
2. **Entradas** — ¿qué colas consume? (Eso te dice a qué hechos reacciona.)
3. **Salidas** — ¿a qué exchanges publica? (Eso te dice qué hechos genera.)
4. **Forma** — ¿tiene HTTP? ¿schedulers? ¿patrones especiales?

Un servicio event-driven se entiende por **lo que escucha y lo que grita**, no por una API.

En nuestro sistema

4.1 — api-gateway

Responsabilidad	Único punto de entrada HTTP. Valida, transforma requests en eventos, publica a RabbitMQ.
Estado	<i>Stateless</i> , sin BD .
Servidor	Undertow .
Forma	Responde 202 / 200 sin bloquear. Productor puro de eventos.

4 casos de uso (endpoint → evento → exchange):

Endpoint	Evento / destino	Exchange	Tipo / routing key
POST /api/v1/payments	OnPaymentRequestReceivedEvent	on_new_payment_request	fanout, rk ""
POST /api/v1/webhooks/{provider}/notifications	notificación de webhook	on_new_webhook_notification	topic, rk = providerCode
POST /api/v1/payments/{uuid}/cancel	solicitud de cancelación	on_new_cancel_order_request	(entrada de cancelación)
POST /api/v1/payments/{uuid}/refund	solicitud de reembolso	on_new_refund_order_request	(entrada de reembolso)

Idea: el gateway es deliberadamente "tonto". No tiene reglas de negocio de pago; su trabajo es **traducir HTTP → evento** lo más rápido posible y soltar la conexión. Por eso usa Undertow (servidor liviano, eficiente con muchas conexiones concurrentes) y no necesita base.

4.2 — payment-service

Responsabilidad	Orquestador central / Aggregate Root. Dueño de la verdad del estado de cada pago.
Estado	PostgreSQL 16 (9 tablas) con HikariCP.
Forma	100% event-driven, sin endpoints HTTP . Consume 9 colas , publica a 6 exchanges .
Patrón clave	Transactional Outbox (lo vemos a fondo en Etapa 5).
Schedulers	ExpiredOrdersScheduler (cada 5s) y RefundInProgressScheduler (cada 60s).

Transactional Outbox, en una frase: cada evento que va a publicar se **persiste en la misma transacción** que la entidad de negocio. Se intenta publicar de una vía un `ApplicationEvent` de Spring; si esa publicación falla, un **scheduler de respaldo reintenta cada 3s (máx 3 intentos)**. Así nunca tenés "guardé el estado pero perdí el evento" (ni al revés). Casos idempotentes con **rollback automático**.

Los schedulers internos son la pieza que rompe la idea de "todo lo dispara un HTTP":

- ExpiredOrdersScheduler (5s): busca órdenes que vencieron sin pagarse y dispara su expiración. **Nadie hace un request para esto:** lo dispara el tiempo.
- RefundInProgressScheduler (60s): revisa reembolsos en curso y los hace avanzar/consultar estado.

Mentalidad: este es el servicio donde más se nota el cambio respecto del monolito. No tiene puerta HTTP; **vive reaccionando a colas y a relojes internos**. Es el cerebro: decide, persiste y emite los próximos eventos.

4.3 — mercadopago-adapter

Responsabilidad	Adaptador de salida (hexagonal) hacia MercadoPago. Encapsula TODA la integración con su API REST.
Estado	<i>Stateless</i> , sin BD .
Forma	Consume 6 colas , publica 5 exchanges .
Integración	POST /v1/orders , GET /v1/orders/{id} , .../cancel , .../refund .
Seguridad	Valida webhooks con firma HMAC-SHA256 . Token y webhook secret por cliente .
Resiliencia	Reintentos automáticos con DLQ . Soporta proxy HTTP corporativo .

Idea: todo lo "sucio" de hablar con un tercero (autenticación por cliente, formato de la API, validación de firmas, reintentos cuando MercadoPago tira error o timeout) queda **encapsulado acá**. El resto del sistema no sabe que MercadoPago existe: solo conoce eventos GWP. Si mañana entra otro proveedor, idealmente nace otro adapter hermano, sin tocar al payment-service.

La **firma HMAC-SHA256** es cómo el adapter verifica que un webhook **vino de verdad de MercadoPago** y no de un impostor: MercadoPago firma el cuerpo con un secreto compartido (el *webhook secret*, distinto por cliente), y el adapter recalcula la firma y la compara. Si no coincide, descarta.

4.4 — notification-service

Responsabilidad	Adaptador de salida HTTP hacia los clientes. Convierte eventos del payment-service en HTTP POST firmados a los webhooks de cada cliente.
Estado	Sin BD .
Forma	Consume 2 colas , publica 1 exchange de auditoría (<i>on_notification_attempt_completed</i>).
Firma	HMAC-SHA256 sobre el POST que manda al cliente.
Resiliencia	Backoff exponencial (Spring Retry + Apache HttpClient 5), configurable por cliente.
Patrón clave	12 extractores de metadatos inyectados por proveedor → Strategy .

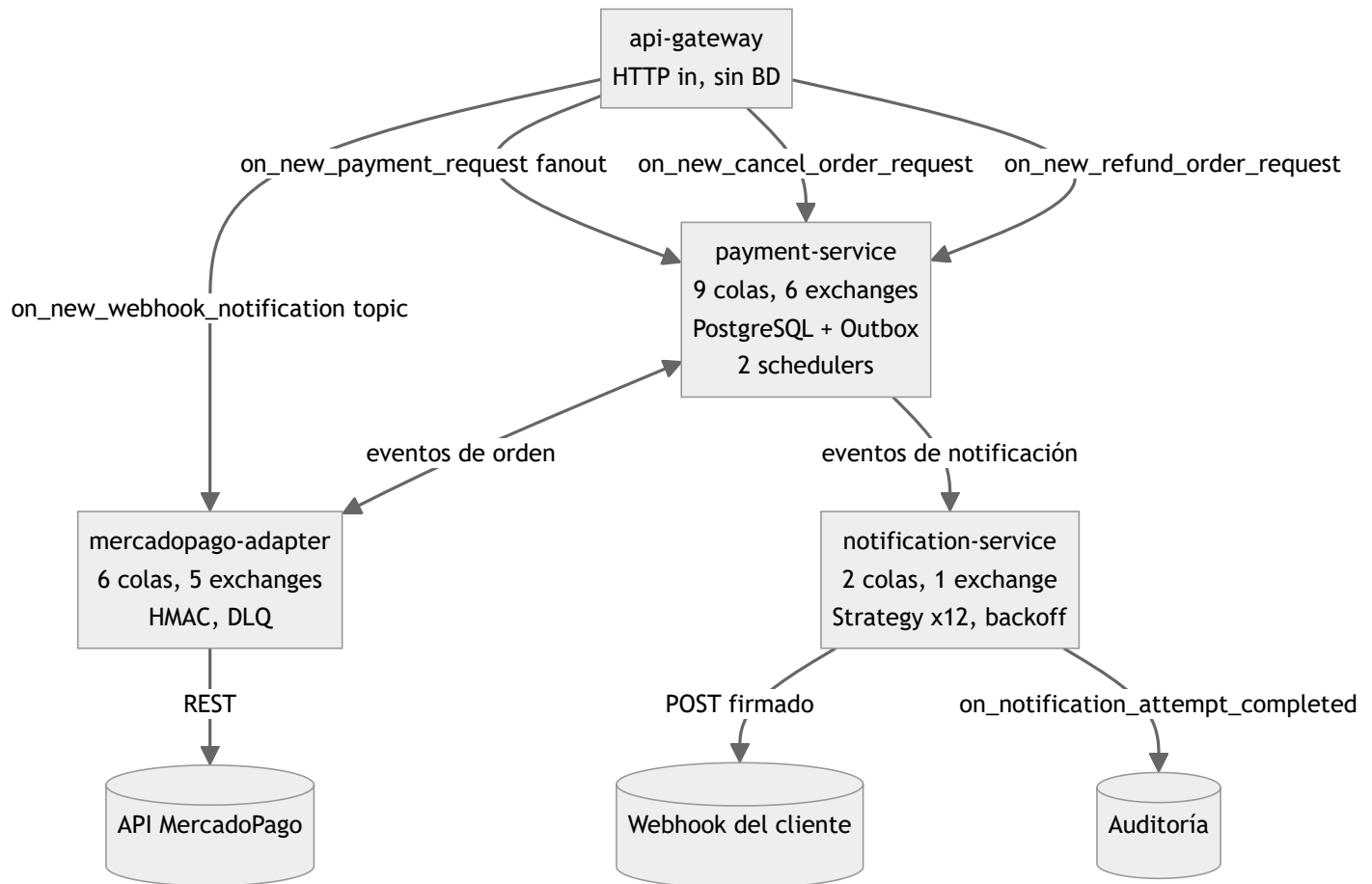
El patrón Strategy de los 12 extractores: distintos proveedores meten la metadata del pago en lugares distintos del payload. En vez de un *if/else* gigante por proveedor, hay **una interfaz "extractor de metadatos" y 12 implementaciones**, una por proveedor. En runtime se elige la del proveedor correspondiente. Eso es Strategy puro: comportamiento intercambiable detrás de una interfaz común.

Analogía: es el mismo patrón que cuando inyectás *List<Validator>* y elegís el que aplica según el tipo. Acá es *List<MetadataExtractor>* (o un *Map<provider, extractor>*) y se selecciona por proveedor.

El backoff exponencial es su forma de no rendirse al primer fallo ni martillar al cliente: si el webhook del cliente no responde, reintentando esperando cada vez más (1s, 2s, 4s, 8s...). Y cada intento — exitoso o no — se **audita** publicando a

`on_notification_attempt_completed`.

Vista integrada



📁 Qué buscar en el código (para la semana)

- **api-gateway:** los **controllers** (4 endpoints) y, en cada uno, la línea donde **publica el evento** y devuelve `202 / 200`. Confirmá que no hay acceso a BD.
- **payment-service:** contá los `@RabbitListener` (esperá 9) y las **publicaciones a exchanges** (6). Buscá la **tabla/entidad del Outbox** (algo tipo `outbox_events`) y los dos schedulers (`@Scheduled`, métodos cada 5s y 60s).
- **mercadopago-adapter:** el **cliente HTTP** de MercadoPago (puerto de salida), la **validación HMAC** del webhook, y la config de **DLQ/reintentos**. Buscá dónde se resuelven **token y secret por cliente**.
- **notification-service:** la **interfaz del extractor de metadatos** y sus **12 implementaciones** (Strategy), la config de **Spring Retry** (backoff), y dónde publica `on_notification_attempt_completed`.

? Preguntas para verificar que entendiste

1. ¿Por qué el api-gateway no tiene base de datos y el payment-service sí? ¿Qué dice eso sobre la responsabilidad de cada uno?
 2. El payment-service no tiene endpoints HTTP. Entonces, ¿de qué dos formas "le llegan cosas para hacer"?
 3. ¿Qué encapsula el mercadopago-adapter que el resto del sistema **no** debería conocer? Dá dos ejemplos.
 4. Explicá el patrón Strategy de los 12 extractores: ¿qué problema evita frente a un `switch` por proveedor?
 5. ¿Para qué publica el notification-service a `on_notification_attempt_completed` si nadie "necesita" ese evento para el flujo principal?
-

Mini-ejercicio

Hacé **cuatro fichas** (una por servicio) en una tarjeta cada una, con solo 4 renglones: **(1)** ¿tiene BD?, **(2)** ¿tiene HTTP?, **(3)** ¿cuántas colas consume?, **(4)** ¿cuántos exchanges publica? Tapá el documento y completá de memoria. Después verificá. El objetivo es que el "esqueleto" del sistema te quede grabado antes de los flujos de la Etapa 6.

Siguiente: `etapa-5-patrones.md`

Etapa 5 — Patrones que tenés que entender sí o sí

Objetivo de la etapa

Dominar los patrones que hacen que un sistema event-driven sea **confiable** y no un caos de mensajes perdidos: **Transactional Outbox, idempotencia, consistencia eventual, entrega at-least-once, reintentos y backoff, DLQ y schedulers**. Para cada uno: qué problema resuelve y dónde aparece en GWP.

Esta es **la etapa más importante del documento**. Si entendés esto, el código después se lee solo.

Teoría

5.1 — El problema raíz: el "dual write"

Imaginá el payment-service procesando un evento. Tiene que hacer **dos cosas**: (a) guardar el nuevo estado en PostgreSQL y (b) publicar un evento a RabbitMQ para que el flujo siga. Son **dos sistemas distintos**. Y acá está la trampa:

```
// EL BUG CLÁSICO — no hagas esto
@Transactional
public void handle(Event e) {
    repo.save(order);           // (a) commit en PostgreSQL
    rabbitTemplate.send(nextEvent); // (b) publish en RabbitMQ
}
```

¿Qué pasa si la app se cae **entre (a) y (b)**? Guardaste el estado pero **nunca publicaste el evento**. El pago quedó "confirmado" en tu base pero el adapter nunca se enteró: flujo cortado, plata en el limbo. Y al revés es peor: si publicás primero y el commit de la base falla, mandaste un evento sobre algo que **no pasó**.

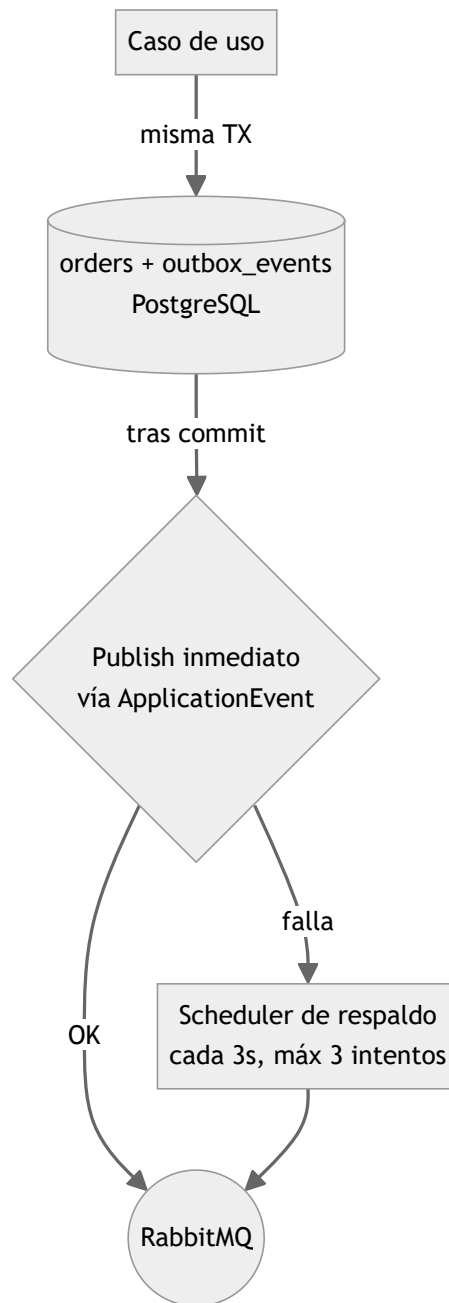
No podés meter PostgreSQL y RabbitMQ en una sola transacción atómica (sin transacciones distribuidas, que son caras y frágiles). Este es el problema del **dual write**, y la solución es el Outbox.

5.2 — Transactional Outbox

La idea es elegante: **convertí el dual write en un single write**. En vez de escribir a dos sistemas, escribí a **uno solo** (PostgreSQL) y dejá que un proceso aparte se encargue de publicar.

Cómo funciona:

1. En la **misma transacción** que guardás la entidad de negocio, insertás una fila en una tabla `outbox_events` con el evento a publicar (serializado). Una sola transacción, atómica: o se guardan **las dos cosas** o **ninguna**.
2. Después de commitear, intentás **publicar de una** (en GWP, vía un `ApplicationEvent` de Spring que dispara la publicación a RabbitMQ tras el commit).
3. Si esa publicación inmediata **falla** (RabbitMQ no responde, red caída), no pasa nada grave: el evento **ya está guardado en** `outbox_events`. Un **scheduler de respaldo reintenta cada 3 segundos (máx 3 intentos)** publicando los eventos pendientes de la tabla.
4. Cuando se publica con éxito, se marca el evento como enviado (o se borra de la tabla).



Por qué es genial: transformaste "guardar estado Y publicar evento, atómicamente" (imposible entre dos sistemas) en "guardar estado Y registrar la intención de publicar, atómicamente" (trivial, es una sola base). La publicación real se vuelve un detalle reintentable que **nunca pierde el evento**.

En GWP: lo implementa el **payment-service**. Tabla `outbox_events`, publish inmediato vía `ApplicationEvent` + scheduler de respaldo cada 3s (máx 3 intentos). Casos idempotentes con **rollback automático**.

5.3 — Entrega at-least-once

Recordá de la Etapa 1: el mensaje no se borra hasta el **ack**. Si el consumidor se cae después de procesar pero antes de ackear, el broker **redan** el mensaje. Conclusión inevitable:

En este tipo de sistemas, un mensaje puede entregarse **más de una vez**. No es un bug: es la garantía *at-least-once*. La alternativa (*at-most-once*) perdería mensajes, y en pagos eso es inaceptable.

Sumale que el Outbox también puede publicar dos veces (publicó, se cayó antes de marcarlo como enviado, el scheduler lo reintenta). Doble motivo para lo que sigue.

5.4 — Idempotencia

Idempotente = procesar el mismo mensaje N veces tiene el **mismo efecto** que procesarlo una vez. Es la **contramedida obligatoria** del at-least-once.

Analogía: `set saldo = 100` es idempotente (lo corras 1 o 5 veces, queda 100). `saldo = saldo + 100` **no lo es** (lo corrés 3 veces y sumaste 300). En pagos, "confirmar la orden #123" tiene que ser idempotente: si el evento de confirmación llega dos veces, la orden se confirma una sola vez, no cobrás dos veces.

Cómo se logra, en general:

- Detectar duplicados por una **clave de negocio** (el `uuid` de la orden, un id de evento): "¿ya procesé esto? entonces no hago nada".
- Diseñar las operaciones como **transiciones de estado idempotentes** ("confirmar" sobre algo ya confirmado = no-op).

En GWP: el contexto menciona explícitamente "**casos idempotentes con rollback automático**" en el payment-service. Es decir: si un evento ya fue procesado (o si su procesamiento viola una invariante), se descarta/revierte sin duplicar efectos.

5.5 — Consistencia eventual

Como cada servicio tiene su propio estado y se comunican por eventos asíncronos, **el sistema no está perfectamente coherente en todo instante**. Hay una ventana — milisegundos a segundos — donde el payment-service ya confirmó el pago pero el notification-service todavía no avisó al cliente. Al rato, todo converge.

El cambio de chip: venís de transacciones ACID donde, al commitar, **todo** está coherente *ya*. Acá aceptás que la coherencia llega *después* ("eventualmente"). A cambio, ganás desacople y disponibilidad. La clave es que el sistema **converge** a un estado correcto, y mientras tanto nunca pierde información (gracias a colas + Outbox + reintentos).

5.6 — Reintentos y backoff

Cuando un consumidor falla por algo **transitorio** (MercadoPago tuvo un timeout, el webhook del cliente devolvió 503), no querés rendirte ni martillar. **Reintentás**, idealmente con **backoff exponencial**: esperarás cada vez más entre intentos (1s, 2s, 4s, 8s...), a veces con un poco de aleatoriedad (*jitter*) para no sincronizar reintentos.

En GWP:

- **mercadopago-adapter:** reintentos automáticos contra la API de MercadoPago, con **DLQ** cuando se agotan.
- **notification-service:** **backoff exponencial** con Spring Retry + Apache HttpClient 5, **configurable por cliente** (cada cliente puede tolerar reintentos distintos).

5.7 — DLQ como red de seguridad

Si un mensaje falla y falla y falla (un *poison message*: payload corrupto, un bug que siempre explota), reintentarlo para siempre **bloquearía la cola** y quemaría recursos. La **DLQ** lo saca de circulación: lo deriva a una cola aparte para inspección humana. El flujo sano sigue; el problemático queda aislado y visible.

Regla de oro operativa: una DLQ que crece es una **alarma**. Significa "hay mensajes que ningún reintento pudo procesar — vení a mirar". En la Etapa 8 vemos cómo investigarla.

5.8 — Schedulers (cuando el disparador es el tiempo)

No todo lo dispara un mensaje o un HTTP. Algunas cosas las dispara **el reloj**. En event-driven los schedulers cumplen dos roles típicos:

1. **Respaldo de entrega:** el scheduler del Outbox (cada 3s) que publica lo que el publish inmediato no pudo.
2. **Reglas de negocio temporales:** cosas que "pasan solas" con el tiempo.

En **GWP**, el payment-service tiene:

- `ExpiredOrdersScheduler` (**cada 5s**): expira órdenes QR que vencieron sin pagarse.
- `RefundInProgressScheduler` (**cada 60s**): hace avanzar/consulta reembolsos en curso.

Analogía: es tu `@Scheduled` de toda la vida. La diferencia es el rol que cumple: acá el scheduler es a la vez **red de seguridad** del mensaje (Outbox) y **fuentes de eventos** (la expiración no la pide nadie; la dispara el tiempo y genera un evento que recorre el sistema).

En nuestro sistema — tabla resumen

Patrón	Dónde vive en GWP	Qué problema resuelve
Transactional Outbox	payment-service (<code>outbox_events</code> , publish + scheduler 3s/máx 3)	dual write: no perder ni inventar eventos
Idempotencia	payment-service (casos idempotentes + rollback)	duplicados por at-least-once
At-least-once	todo el sistema (ack/nack de RabbitMQ)	no perder mensajes
Consistencia eventual	todo el sistema	desacople sin transacción distribuida
Reintentos + DLQ	mercadopago-adapter	fallas transitorias de MercadoPago
Backoff exponencial	notification-service (Spring Retry, por cliente)	webhooks de cliente caídos/lentos
Schedulers	payment-service (Outbox 3s, expiración 5s, refunds 60s)	respaldo de entrega + reglas temporales

Qué buscar en el código (para la semana)

- La **entidad/tabla del Outbox** (`outbox_events` o similar) y la clase que **inserta el evento en la misma transacción** que la entidad. Busca el uso de un `ApplicationEvent` de Spring publicado **tras el commit** (mirá `@TransactionalEventListener` con `phase = AFTER_COMMIT`, o un `ApplicationEventPublisher`).
- El **scheduler del Outbox** (un `@Scheduled` cada ~3s que recorre eventos pendientes) y la lógica de **máx 3 intentos**.
- La **detección de idempotencia**: dónde se chequea "¿ya procesé este evento/uuid?" antes de aplicar efectos.
- En el adapter, la config de **reintentos + DLQ** (argumentos `x-dead-letter-*`, política de retry).
- En el notification-service, la config de **Spring Retry / backoff** (anotaciones `@Retryable` / `@Backoff` o config programática) y dónde se lee la política **por cliente**.
- Los dos `@Scheduled` del payment-service (5s y 60s).

? Preguntas para verificar que entendiste

1. Explicá el problema del **dual write** con tus palabras. ¿Por qué un `@Transactional` que guarda en la base y publica a RabbitMQ **no** lo resuelve?
2. ¿Cómo convierte el Outbox un "dual write" en un "single write"? ¿Qué rol cumple el scheduler de respaldo?
3. ¿Por qué la entrega *at-least-once* **obliga** a que los consumidores sean idempotentes? Dá un ejemplo de operación idempotente y uno que no lo sea en un contexto de pagos.
4. ¿Qué es la consistencia eventual y por qué es un costo aceptable acá? ¿En qué momento del flujo de pago se nota esa ventana?
5. ¿Cuándo un mensaje termina en la DLQ y por qué una DLQ que crece es una señal de alarma?

Mini-ejercicio

Trazá el Outbox a mano. Dibujá la línea de tiempo del payment-service procesando un evento, con estos hitos: **TX abre** → **save(order)** → **insert(outbox_events)** → **TX commit** → **publish inmediato** → **¿OK?**. Ahora **rompé el sistema en 3 lugares distintos** (caída antes del commit, caída justo después del commit pero antes del publish, fallo del publish inmediato) y para cada caso escribí: **¿qué quedó en la base?** y **¿quién termina publicando el evento?**. La conclusión que buscás: en ningún caso se pierde el evento.

Siguiente: `etapa-6-flujos-end-to-end.md`

Etapá 6 — Flujos end-to-end

Objetivo de la etapa

Seguir un mensaje **paso a paso, salto por salto, por los 4 servicios**, en los 5 flujos que existen en GWP. Al terminar deberías poder responder, para cualquier flujo: *¿qué evento se emite, a qué exchange, con qué routing key, a qué cola llega y qué caso de uso lo consume?*

Nota de honestidad: los **nombres de los 4 exchanges de entrada** (`on_new_payment_request` , `on_new_webhook_notification` , `on_new_cancel_order_request` , `on_new_refund_order_request`) y el de auditoría (`on_notification_attempt_completed`) son reales. Los **exchanges/colas intermedios** entre `payment-service` ↔ `adapter` ↔ `notification` **no están nombrados** en la doc de arquitectura, así que acá los describo por su **función** (ej. "evento de orden creada"), no con un nombre inventado. Confirmá los nombres exactos en la clase de configuración de RabbitMQ cuando tengas el repo.

Cómo leer un flujo

Cada salto entre servicios tiene **siempre** la misma anatomía. Entrenate a identificarla:

```
[Servicio A] --emite--> EVENTO --a--> EXCHANGE (tipo, routing key) --binding--> COLA --consume--> [Servicio B / caso de uso]
```

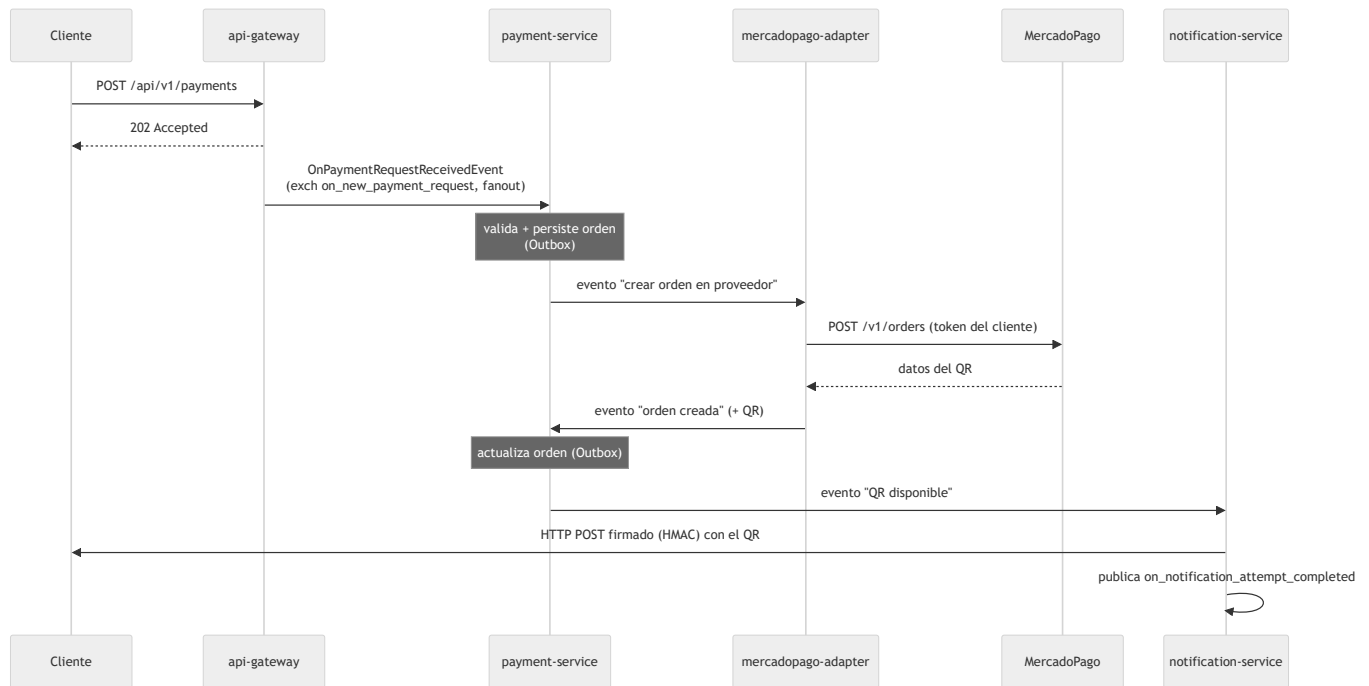
Si para cada salto podés completar esos casilleros, entendiste el flujo. Vamos.

Flujo 1 — Solicitud de pago → generación de QR

El cliente quiere cobrar; el sistema genera una orden QR en MercadoPago y se la devuelve (asíncronamente) al cliente.

Paso a paso:

1. **Cliente** → **api-gateway**: `POST /api/v1/payments` . El gateway valida, arma `OnPaymentRequestReceivedEvent` , lo publica a `on_new_payment_request` (fanout, rk `"`) y responde `202` .
2. **api-gateway** → **payment-service**: el evento llega a la cola del `payment-service`. Su caso de uso "procesar solicitud de pago" valida con el dominio, **persiste la orden** (estado inicial) y, vía **Outbox**, emite un evento "hay que crear la orden en el proveedor".
3. **payment-service** → **mercadopago-adapter**: el adapter consume ese evento, llama a `POST /v1/orders` de MercadoPago (con el token del cliente), recibe los datos del QR y **emite un evento "orden creada en el proveedor"** (con el QR/datos).
4. **mercadopago-adapter** → **payment-service**: el `payment-service` consume la respuesta, **actualiza la orden** con el QR (misma mecánica Outbox) y emite un evento "orden lista / QR disponible".
5. **payment-service** → **notification-service**: el `notification-service` consume ese evento, arma un **HTTP POST firmado (HMAC)** y lo manda al webhook del cliente con el QR. Publica el resultado a `on_notification_attempt_completed` (auditoría).



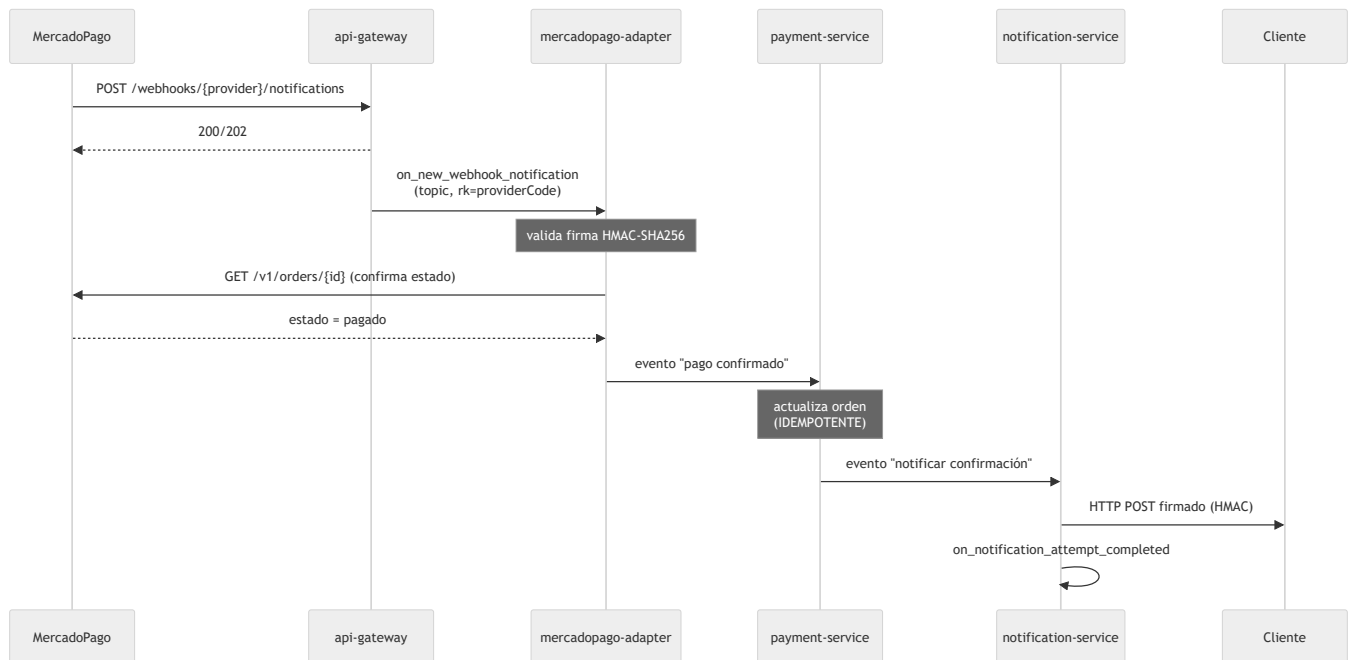
Fijate que el cliente recibió su `202` en el paso 1, muchísimo antes de que el QR estuviera listo. Cuando el QR está, se entera **por su webhook**. Eso es asincronía pura.

🔍 Flujo 2 — Confirmación de pago (webhook de MercadoPago, happy path)

El usuario final pagó el QR. MercadoPago avisa por webhook; el sistema confirma y notifica.

Paso a paso:

- MercadoPago → api-gateway:** `POST /api/v1/webhooks/{provider}/notifications`. El gateway publica a `on_new_webhook_notification` (topic, rk = `providerCode`) y responde rápido.
- api-gateway → mercadopago-adapter:** la notificación rutea (por `providerCode`) a la cola del adapter. El adapter **valida la firma HMAC-SHA256** del webhook con el `secret` del cliente. Si es válida, normalmente **consulta el estado real** con `GET /v1/orders/{id}` (no confía solo en el aviso) y emite un evento "pago confirmado".
- mercadopago-adapter → payment-service:** el payment-service consume "pago confirmado", **actualiza la orden a confirmada** de forma **idempotente** (si el webhook llega dos veces, confirma una sola), y emite un evento "pago confirmado al cliente".
- payment-service → notification-service:** notifica al cliente con POST firmado y audita el intento.



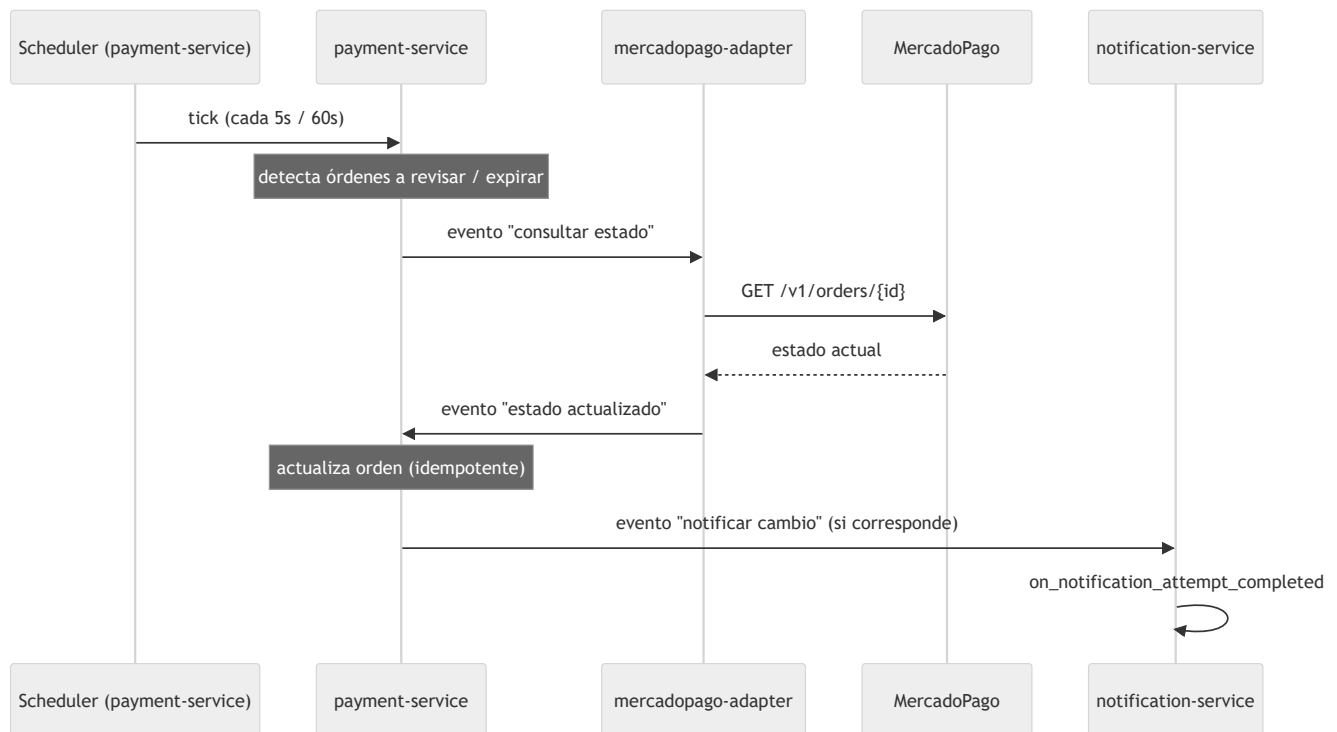
El detalle de oro: el paso 3 es idempotente *a propósito*. MercadoPago puede mandar el webhook más de una vez, y RabbitMQ puede redar el mensaje (at-least-once). El pago se confirma **una sola vez**.

🔍 Flujo 3 — Consulta de cambio de estado (disparada por scheduler)

Este flujo **no empieza con un HTTP**. Lo dispara el **tiempo**. Es el ejemplo perfecto de "fuente de eventos = reloj".

Paso a paso:

1. **Scheduler (payment-service):** el `RefundInProgressScheduler` (cada 60s) o la lógica de polling de estado detecta órdenes que necesitan que se consulte su estado real en el proveedor, y emite un evento "consultar estado de la orden". (El `ExpiredOrdersScheduler`, cada 5s, es la variante que **expira** órdenes vencidas y emite "orden expirada".)
2. **payment-service** → **mercadopago-adapter:** el adapter consume, hace `GET /v1/orders/{id}` y emite un evento con el estado actualizado.
3. **mercadopago-adapter** → **payment-service:** el payment-service actualiza la orden (idempotente) y, si corresponde, emite un evento para **notificar el cambio al cliente**.
4. **payment-service** → **notification-service:** notifica y audita.



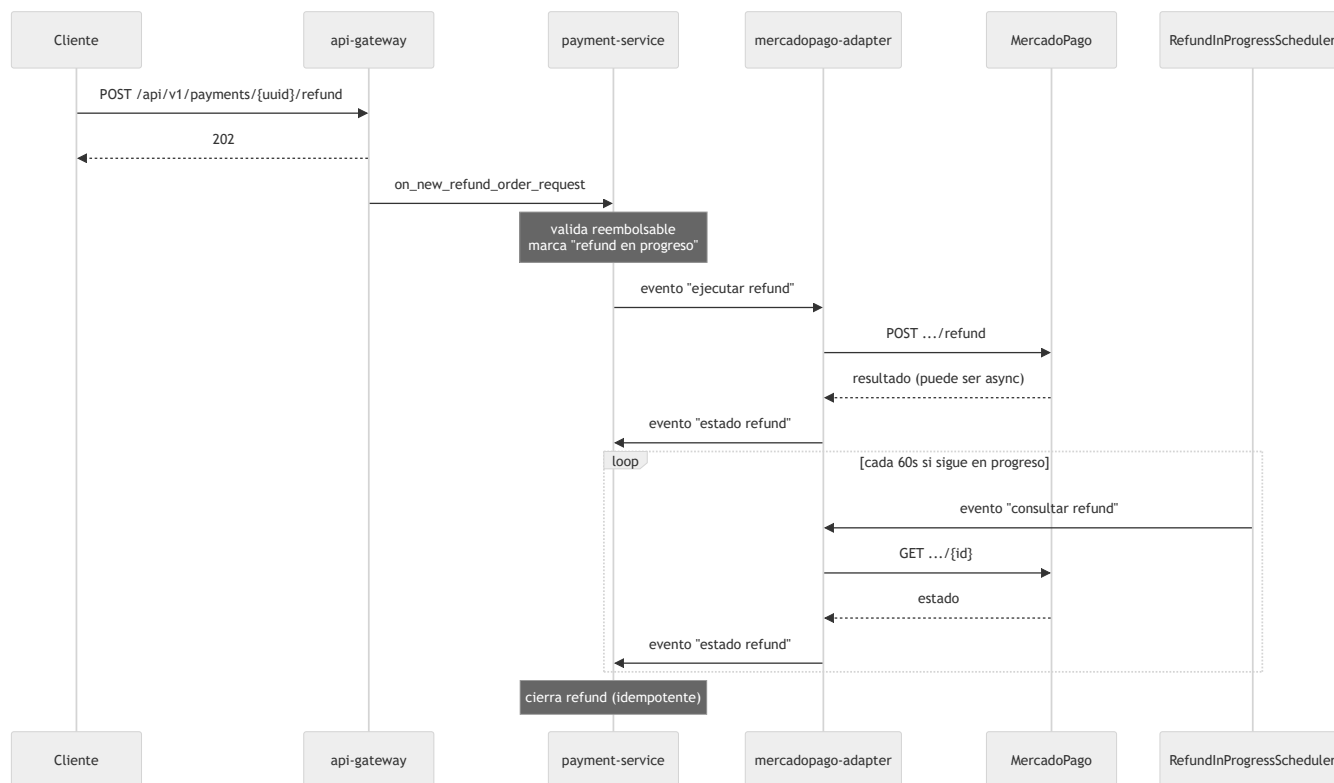
Lo nuevo para vos: acá no hay request del cliente. El sistema "se mueve solo". En un monolito esto sería un `@Scheduled` que llama un service; acá el scheduler **emite un evento** y deja que el flujo event-driven haga el resto.

🔍 Flujo 4 — Reembolso de pago

El cliente pide devolver un pago ya confirmado.

Paso a paso:

1. **Cliente** → **api-gateway**: `POST /api/v1/payments/{uuid}/cancel` ... no — reembolso es `POST /api/v1/payments/{uuid}/refund`. El gateway publica a `on_new_refund_order_request` y responde rápido.
2. **api-gateway** → **payment-service**: consume la solicitud, valida con el dominio que **la orden sea reembolsable** (regla de negocio: tiene que estar confirmada, etc.), marca la orden "reembolso en progreso" y emite un evento "ejecutar reembolso en proveedor".
3. **payment-service** → **mercadopago-adapter**: el adapter llama a `.../refund` de MercadoPago y emite el resultado.
4. **mercadopago-adapter** → **payment-service**: actualiza el estado del reembolso. Acá entra el `RefundInProgressScheduler (60s)`: si el reembolso queda "en progreso" (MercadoPago a veces los procesa async), el scheduler lo hace avanzar consultando estado hasta cerrarlo.
5. **payment-service** → **notification-service**: notifica el reembolso al cliente y audita.

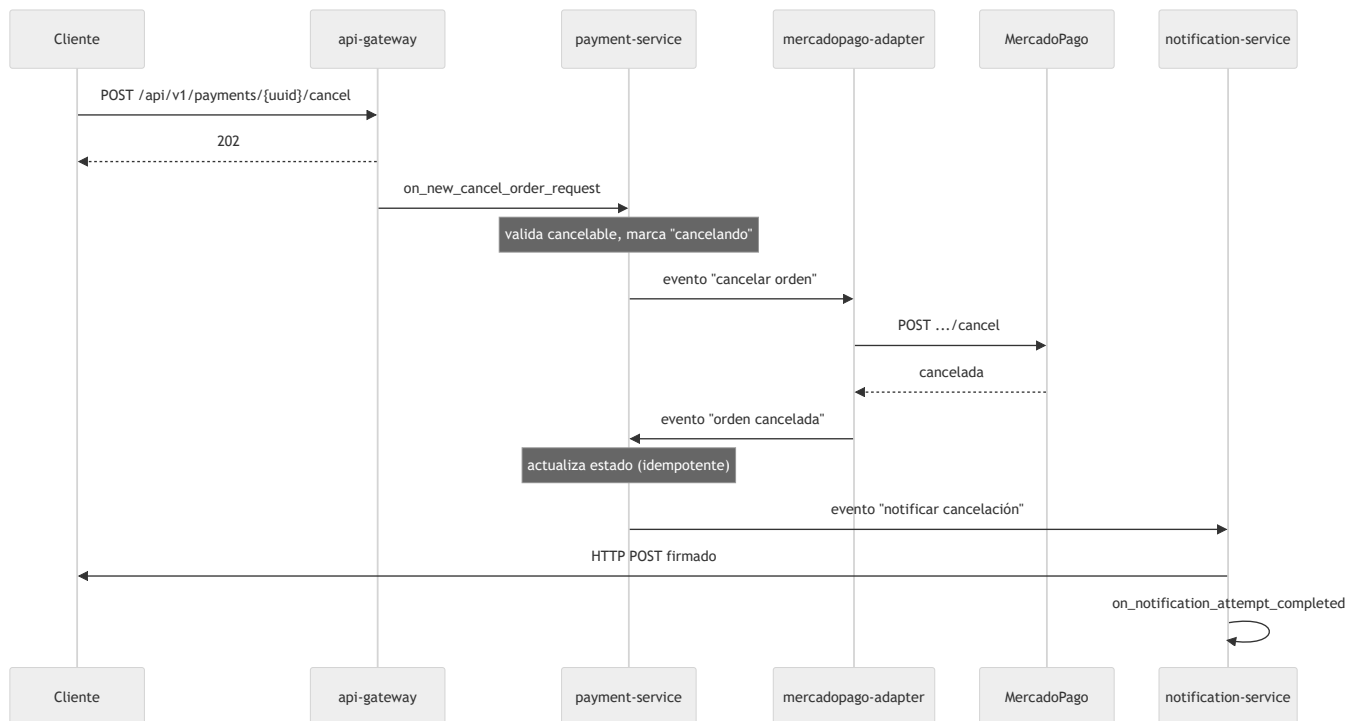


🔍 Flujo 5 — Cancelación de orden

El cliente cancela una orden **antes** de que se pague (típicamente, una orden QR todavía pendiente).

Paso a paso:

1. **Cliente** → **api-gateway**: POST `/api/v1/payments/{uuid}/cancel`. Publica a `on_new_cancel_order_request` y responde rápido.
2. **api-gateway** → **payment-service**: valida que la orden **sea cancelable** (no pagada aún), marca "cancelando" y emite "cancelar orden en proveedor".
3. **payment-service** → **mercadopago-adapter**: llama a `.../cancel` de MercadoPago y emite el resultado.
4. **mercadopago-adapter** → **payment-service**: actualiza la orden a "cancelada" (idempotente) y emite "notificar cancelación".
5. **payment-service** → **notification-service**: notifica y audita.



Relación con el Flujo 3: una orden que **nadie cancela ni paga** termina expirando sola gracias al `ExpiredOrdersScheduler` (5s). Cancelación = el cliente lo pide; expiración = lo dispara el reloj. Mismo destino (orden cerrada), distinto disparador.

📁 Qué buscar en el código (para la semana)

- Para cada flujo, arrancá por el **caso de uso de entrada en el payment-service** (el `@RabbitListener` que consume el primer evento) y seguí **qué evento publica** al final. Eso te da el "siguiente salto".
- Cruzá cada **publicación** con el **binding** correspondiente en la config de RabbitMQ para confirmar exchange + routing key + cola destino.
- En el adapter, ubicá las 4 llamadas REST (`POST /v1/orders` , `GET /v1/orders/{id}` , `.../cancel` , `.../refund`) y qué evento dispara cada respuesta.
- Confirmá los **nombres reales** de los exchanges/colas intermedios que acá describí por función.

? Preguntas para verificar que entendiste

1. En el Flujo 1, ¿en qué momento exacto el cliente recibe respuesta del gateway, y cómo se entera después de que el QR está listo?
2. En el Flujo 2, ¿por qué el adapter hace `GET /v1/orders/{id}` en vez de confiar directamente en el contenido del webhook?
3. El Flujo 3 no tiene un actor "Cliente" al inicio. ¿Quién lo dispara y qué lo hace distinto de los demás?
4. En el Flujo 4, ¿qué rol cumple el `RefundInProgressScheduler` y por qué hace falta un loop de consulta?
5. Para **cualquier** salto entre dos servicios, ¿cuáles son los 5 casilleros que tenés que poder completar?

Mini-ejercicio

Elegí el **Flujo 2** (confirmación de pago). En una hoja, listá **cada cola que el mensaje atraviesa** desde que MercadoPago manda el webhook hasta que el cliente recibe la notificación, y **contá cuántas son**. Para cada cola, anotá qué servicio la consume. Después marcá **los dos puntos del flujo donde la idempotencia es indispensable** y explicá en una línea qué saldría mal sin ella.

Siguiente: `etapa-7-infraestructura.md`

Etapas 7 — Infraestructura y "verlo correr"

Objetivo de la etapa

Entender **dónde y cómo corre** GWP: contenedores, Kubernetes/Minikube, el namespace `dev`, y las piezas (pods, deployments, services, configmaps, secrets). Y, sobre todo, tener una **guía concreta** para levantarlo localmente, mandar una solicitud de prueba y **ver los mensajes pasando por RabbitMQ**.

Esta etapa es la más "manos" del documento. Cuando tengas acceso al cluster, volvé acá con la compu al lado.

 **Si Docker/Kubernetes te suenan nuevos:** esta etapa te enseña a **operar** el sistema (mirar pods, abrir la UI, ver logs). Para entender la **maquinaria por dentro** — imagen vs contenedor, cómo un Service encuentra a sus pods (labels/selectors), DNS interno, ConfigMap/Secret, PV/PVC, la jerarquía Deployment→ReplicaSet→Pod y los gotchas de Minikube — leé el **Apéndice A — Fundamentos de Docker y Kubernetes**. Esta etapa asume esos conceptos; el apéndice los construye desde cero.

Teoría

Contenedores, rapidísimo

Venís de desplegar `.jar` / `.war` en un servidor con un Tomcat instalado. Un **contenedor** empaqueta tu app + **todo lo que necesita** (runtime, libs, config) en una unidad que corre igual en cualquier lado. En GWP, además, los servicios se compilan a **binario nativo con GraalVM**, así que el contenedor lleva un ejecutable que arranca en milisegundos y consume poca RAM — ideal para correr varios pods chicos.

Analogía: si el `.jar` era "te paso el plato y vos poné la mesa (JVM, config, etc.)", el contenedor es "te paso la vianda lista, abrila y comé". Misma vianda en tu máquina, en CI y en el cluster.

Kubernetes en 6 sustantivos

Kubernetes (K8s) **orquesta** contenedores: decide en qué máquina corre cada uno, los reinicia si se caen, los conecta entre sí. GWP corre sobre **Minikube** (un K8s de **1 nodo** para desarrollo local). Los conceptos que vas a tocar:

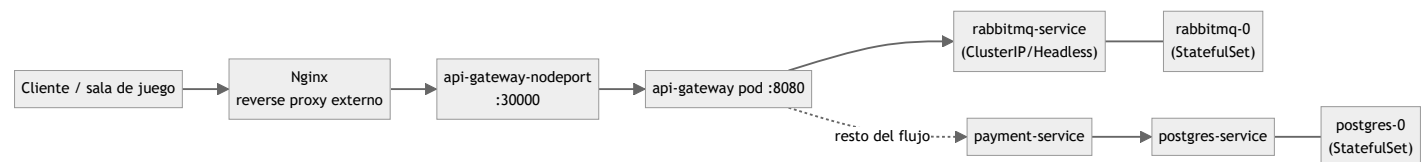
Concepto	Qué es	Analogía
Pod	la unidad que corre tu contenedor (1 o más)	un proceso corriendo
Deployment	declara "quiero N réplicas de este pod" y las mantiene	un supervisor que reinicia procesos caídos
Service	un nombre/IP estable para llegar a los pods (que van y vienen)	un DNS interno / balanceador
ConfigMap	configuración no secreta (URLs, flags) inyectada al pod	un <code>application.yml</code> externo
Secret	configuración sensible (tokens, secrets de webhook, password de PG)	una bóveda de credenciales
Namespace	una "carpeta" lógica que aísla recursos	un environment (<code>dev</code>)

En GWP: todo vive en el namespace `dev` — por eso **todos los comandos llevan** `-n dev`. Cada microservicio es un **Deployment con 1 réplica**. RabbitMQ y PostgreSQL no son deployments comunes sino **StatefulSets** (`rabbitmq-0`, `postgres-0`), porque tienen **estado/identidad estable** (un StatefulSet le da a cada pod un nombre fijo y almacenamiento persistente — clave para una base o un broker).

Tipos de Service que vas a ver

- **NodePort** — expone un puerto del nodo hacia afuera. El **api-gateway** se expone así: `api-gateway-nodeport :30000` → `:8080` del pod.
- **ClusterIP** — solo accesible *dentro* del cluster (la mayoría de los servicios internos).
- **Headless** — sin IP propia; se usa con StatefulSets para resolver cada pod por nombre (`rabbitmq-0`, `postgres-0`).

Cómo entra el tráfico



Nginx es el reverse proxy externo: recibe el tráfico de afuera y lo manda al NodePort del gateway (`:30000`), que adentro llega al `:8080` del pod. De ahí en más, todo es RabbitMQ.

Las dos piezas con estado

- **RabbitMQ** — el broker. AMQP en `5672` (por donde hablan los servicios) y **UI Management en 15672** (por donde mirás vos). StatefulSet `rabbitmq-0`.
- **PostgreSQL 16** — base `payment_db` en `5432`, usada **solo** por el `payment-service`. StatefulSet `postgres-0`.



En nuestro sistema — guía práctica

1. Ver qué está corriendo

```

kubectl get pods -n dev      # ¿están todos los pods Running?
kubectl get svc -n dev       # los services y sus puertos/tipos

```

Esperás ver, como mínimo: los 4 microservicios, `rabbitmq-0` y `postgres-0`. Si alguno está en `CrashLoopBackOff` o `Pending`, ahí empieza el troubleshooting (Etapa 8).

2. Abrir la UI de RabbitMQ (tu ventana al sistema)

```
kubectl port-forward -n dev svc/rabbitmq-service 15672:15672
# abrí http://localhost:15672 (user/pass suelen estar en un Secret/ConfigMap)
```

En la UI:

- **Exchanges:** confirmá tipos (fanout/topic/direct) y bindings.
- **Queues:** mirá la **profundidad** de cada cola (mensajes encolados). Esto es lo que más vas a usar: una cola que crece = consumidor lento o caído.
- Buscá la **DLQ**: si tiene mensajes, algo falló más de la cuenta.

3. Mandar una solicitud de prueba y observar

Idea del experimento (la secuencia exacta de comandos la ajustás con el repo):

1. Dejá abierta la UI de RabbitMQ en la vista de **Queues**.
2. Si el gateway no es accesible directo, exponelo: `kubectl port-forward -n dev svc/api-gateway-... 8080:8080` (o usá el `NodePort :30000` vía Minikube).
3. Mandá un `POST /api/v1/payments` (con `curl` /Postman) con un body de prueba.
4. **Mirá la magia:** verás un pico de tráfico en la cola del `payment-service`, después en la del `adapter`, después la del `notification-service`. El mensaje **viaja**. Eso que en la Etapa 6 dibujaste como `sequenceDiagram`, acá lo ves moverse en vivo.

4. Mirar logs

```
kubectl logs -n dev <pod-del-payment-service> -f      # -f = seguir en vivo
kubectl logs -n dev <pod-del-adapter> -f
```

Seguir los logs de los 4 servicios en paralelo mientras mandás un pago es **la mejor forma de ver el flujo event-driven con tus ojos**: ves cada servicio "despertarse" cuando le llega su evento.

5. Panorama visual (opcional)

```
minikube dashboard # UI web con pods, deployments, services, etc.
```

Comandos de bolsillo

```
kubectl get pods -n dev
kubectl get svc -n dev
kubectl describe pod -n dev <pod>      # eventos/errores de arranque
kubectl logs -n dev <pod> -f
kubectl port-forward -n dev svc/rabbitmq-service 15672:15672
minikube dashboard
```

Qué buscar en el código / repo (para la semana)

- Los **manifiestos de Kubernetes** (carpeta tipo `k8s/`, `deploy/`, `manifests/`, o `.yaml` con `kind: Deployment / Service / StatefulSet / ConfigMap / Secret`). Ahí confirmás puertos, réplicas y nombres reales de services.
- El **ConfigMap** y el **Secret**: qué configuración recibe cada servicio (URL de RabbitMQ, datos de PostgreSQL, tokens y webhook secrets por cliente).
- El o los **Dockerfile** (probablemente *multi-stage* con build nativo GraalVM): confirma la compilación a binario nativo.
- La config de **Nginx** (reverse proxy) si está versionada.
- **Confirmá la versión de Java/Spring Boot** en el archivo de build (`pom.xml` → `<java.version>`, o `build.gradle`). Recordá: la doc dice **Java 21 / Spring Boot 3.5**, vos sospechabas Java 23 — este es el lugar donde lo verificás.

Preguntas para verificar que entendiste

1. ¿Por qué RabbitMQ y PostgreSQL son **StatefulSets** (`rabbitmq-0`, `postgres-0`) y no Deployments comunes como los microservicios?
2. ¿Cuál es la diferencia entre un Service **NodePort** y uno **ClusterIP**, y por qué el api-gateway usa NodePort?
3. ¿Para qué sirve `kubect1 port-forward -n dev svc/rabbitmq-service 15672:15672` y qué vas a hacer una vez conectado?
4. Si entrás a la UI de Management y la cola del payment-service crece sin parar mientras la del adapter está en 0, ¿qué sospechás y qué comando correrías para confirmar?
5. ¿Dónde viven los **tokens y webhook secrets por cliente** en este esquema de infraestructura, y por qué no en un ConfigMap?

Mini-ejercicio

Escribí en una hoja la **secuencia exacta de comandos** que correrías, en orden, para: (1) confirmar que todos los pods están arriba, (2) abrir la UI de RabbitMQ, (3) mandar un pago de prueba, y (4) seguir los logs del payment-service mientras lo hacés. No los ejecutes todavía (no tenés acceso aún): el objetivo es que el "runbook" te quede listo para el día que entres al cluster.

Siguiente: `etapa-8-consolidar.md`

Etapas 8 — Consolidar

Objetivo de la etapa

Cerrar el círculo. Usar todo lo aprendido en un **ejercicio mental de diseño** ("cómo agregaría un proveedor nuevo"), tener a mano un **mapa de troubleshooting** para los problemas típicos de un sistema event-driven, y un **glosario de una página** para repasar de un vistazo.

Ejercicio de cierre: "¿Cómo agregaría un proveedor nuevo?"

Esta es la pregunta que demuestra si entendiste la arquitectura. Hoy GWP habla con MercadoPago. Imaginá que entra un proveedor nuevo — digamos "PagoYa". ¿Qué tocás?

Pensalo vos primero, en una hoja, antes de leer lo de abajo. Recorré los 4 servicios y preguntate "¿este se entera de que existe PagoYa?".

La respuesta (y por qué la arquitectura te ayuda)

- 1. api-gateway — casi sin cambios.** El endpoint de webhooks ya es genérico: `POST /api/v1/webhooks/{provider}/notifications`. El `{provider}` se vuelve la **routing key** (`providerCode`) en el exchange `on_new_webhook_notification` (topic). Para que las notificaciones de PagoYa ruteen a su adapter, alcanza con **agregar un binding** con la routing key de PagoYa. **No tocás el código del gateway.** ← *Acá ves por qué el exchange de webhook es `topic` y no `fanout`.*
- 2. Un nuevo `pagoya-adapter` — el grueso del trabajo.** Nace un adapter hermano del `mercadopago-adapter`: consume las colas de PagoYa, traduce eventos GWP ↔ API de PagoYa, valida sus webhooks con **su** esquema de firma, maneja **sus** tokens/secrets por cliente, y sus reintentos/DLQ. Como el adapter **encapsula** todo lo del proveedor, el resto del sistema ni se entera del cambio.
- 3. `payment-service` — poco o nada.** Es el *Aggregate Root* y razona en **eventos de dominio genéricos** ("orden creada", "pago confirmado"), no en MercadoPago. Mientras el `pagoya-adapter` emita/consuma los mismos eventos de dominio, el `payment-service` sigue igual. (*Acá ves el valor de hexagonal: el negocio no conoce al proveedor.*)
- 4. `notification-service` — una Strategy nueva.** Recordá los **12 extractores de metadatos** (patrón Strategy). Si PagoYa mete la metadata distinto, agregás **una implementación más** del extractor para PagoYa. No tocás las otras 12. (*Acá ves el valor de Strategy.*)

La moraleja: una arquitectura event-driven + hexagonal + Strategy hace que "agregar un proveedor" sea **sumar piezas**, no **editar el corazón**. Si tu respuesta hubiera sido "toco el `payment-service` para meter un `if (provider == pagoya)`", la arquitectura estaría mal — y justamente está diseñada para que no haga falta.

Troubleshooting típico

Tres síntomas clásicos de sistemas event-driven y cómo encararlos. **Regla general: el estado del sistema se lee en RabbitMQ (colas/DLQ) y en los logs de los pods.**

Síntoma 1 — "Hay mensajes en la DLQ"

Qué significa: uno o más mensajes fallaron lo suficiente como para ser desviados (poison message, payload inesperado, bug que siempre explota, o un dependiente caído mucho tiempo).

Cómo investigar:

1. En la UI de Management → la **DLQ** → mirá un mensaje (payload + headers; suele venir el motivo del descarte en los headers `x-death`).
2. Cruzá con los **logs del consumidor** que la alimenta (`kubectl logs -n dev <pod> -f`) en la ventana de tiempo del fallo.
3. Decidí: ¿es un bug (fix + reproceso) o un dato inválido (descartar)? Reprocesar suele ser re-publicar el mensaje a la cola original una vez corregida la causa.

Síntoma 2 — "Una cola crece y no baja" (consumidor caído/lento)

Qué significa: los mensajes entran pero no se consumen al mismo ritmo.

Cómo investigar:

1. UI de Management → **Queues** → mirá *messages ready* (encolados) vs *unacked* (en vuelo) y las tasas.
2. `kubectl get pods -n dev` → ¿el consumidor está `Running` ? ¿ `CrashLoopBackOff` ?
3. `kubectl logs -n dev <pod>` → ¿está lanzando excepciones (nack en loop)? ¿está lento (esperando a un tercero)?
4. ¿ `unacked` alto y estable? El consumidor tomó mensajes pero no los procesa: está colgado procesando. ¿ `ready` alto y `unacked` 0? No hay consumidor vivo.

Síntoma 3 — "Un evento no llega / el flujo se cortó"

Qué significa: publicaste (o creés que publicaste) pero el siguiente servicio no reacciona.

Cómo investigar (seguí el camino del mensaje):

1. ¿El productor **publicó**? Mirá sus logs y, si aplica, la tabla `outbox_events` : ¿quedó un evento sin marcar como enviado? (entonces el problema es la publicación, no el consumo). El scheduler de respaldo (3s/máx 3) debería reintentar.
2. ¿El exchange tiene el **binding** correcto a la cola? Un binding mal armado (routing key equivocada en un topic) hace que el mensaje se publique **y se pierda** sin llegar a ninguna cola. Verificá en la UI → Exchanges → Bindings.
3. ¿La cola destino existe y tiene consumidor? (Síntoma 2.)
4. ¿Quedó en la **DLQ**? (Síntoma 1.)

El reflejo correcto: ante cualquier problema, recorré el camino que dibujaste en la Etapa 6 — productor → exchange → binding → cola → consumidor — y en cada salto preguntá "¿llegó hasta acá?". El primer salto donde la respuesta es "no" es tu problema.

Glosario de una página (para imprimir y pegar al lado del monitor)

Término	En una línea
Evento	Hecho de negocio que ya pasó; inmutable, en pasado (<code>OnPaymentRequestReceivedEvent</code>).
Command	Orden de "hacé esto"; puede rechazarse. En GWP, <code>record</code> inmutable.
Query	Pregunta sin efectos; lee estado.
Mensaje	El "sobre" (JSON + metadatos) que transporta un evento por el broker.
Exchange	La central de correo: recibe del productor y rutea a colas.
Queue (cola)	Casillero durable donde el mensaje espera al consumidor.
Binding	Regla que conecta exchange ↔ cola (con o sin patrón de routing key).
Routing key	Etiqueta del mensaje que el exchange usa para rutear (en webhook = <code>providerCode</code>).
fanout	Exchange que copia a TODAS las colas atadas; ignora routing key.
topic	Exchange que rutea por patrón de routing key (<code>*</code> , <code>#</code>).
direct	Exchange que rutea por routing key exacta.
Productor / Consumidor	Quien publica / quien lee mensajes.
ack / nack	"Procesado OK, borralo" / "falló, re-encolá o mandá a DLQ".
Prefetch	Cuántos mensajes sin-ack tiene un consumidor en vuelo a la vez.
DLQ	Cola de descarte para mensajes que fallan repetidamente; red de seguridad.
at-least-once	Garantía: el mensaje llega ≥ 1 vez (puede duplicarse).
Idempotencia	Procesar N veces = mismo efecto que 1 vez. Contramedida del at-least-once.
Consistencia eventual	El sistema converge a un estado coherente <i>al rato</i> , no al instante.
Transactional Outbox	Persistir el evento en la misma TX que la entidad; publicar después (con reintento).
Dual write	El problema de escribir a base + broker sin atomicidad; lo resuelve el Outbox.
Backoff exponencial	Reintentar esperando cada vez más (1s, 2s, 4s...).
Hexagonal (Ports & Adapters)	El dominio define puertos; el mundo exterior los implementa. Dependencias hacia adentro.
Puerto / Adapter	Interface que el negocio necesita / implementación concreta (JPA, HTTP, RabbitMQ).
CQRS	Separar comandos (escriben) de queries (leen).
Aggregate Root	Dueño de la consistencia de un agregado; en GWP, el payment-service.
Strategy	Una interfaz, muchas implementaciones intercambiables (los 12 extractores).
StatefulSet	Recurso K8s para pods con identidad/almacenamiento estable (<code>rabbitmq-0</code> , <code>postgres-0</code>).
NodePort / ClusterIP / Headless	Service expuesto al nodo / interno al cluster / sin IP (para StatefulSet).
ConfigMap / Secret	Config no sensible / sensible inyectada a los pods.

Qué buscar en el código (para la semana)

- Para validar el ejercicio del proveedor nuevo: compará la **estructura del** `mercadopago-adapter` con cómo se conectan los **eventos de dominio** en el `payment-service`. Confirmá que el `payment-service` **no menciona "mercadopago"** en su lógica de negocio.
- Ubicá la **interfaz del extractor de metadatos** y contá sus implementaciones (deberían ser 12). Ese es el punto de extensión por proveedor en el `notification-service`.
- Revisá la config de **DLQ** y de `outbox_events` otra vez, ahora con la mira puesta en *cómo investigarías* un problema real.

Preguntas para verificar que entendiste

1. Para agregar el proveedor "PagoYa", ¿qué tocás en cada uno de los 4 servicios? ¿Cuál es el que más cambia y cuál casi no cambia, y por qué?
2. ¿Por qué el exchange de webhooks **tiene que ser** `topic` (y no `fanout`) para que el ejercicio del proveedor nuevo funcione limpio?
3. Ves mensajes en la DLQ. Enumerá los pasos para investigar la causa.
4. Una cola tiene `ready` alto y `unacked` en 0. ¿Qué está pasando? ¿Y si fuera `unacked` alto y estable?
5. Un evento "no llega" al siguiente servicio. ¿Cuál es el camino que recorrés salto por salto para encontrar dónde se cortó?

Mini-ejercicio (cierre del curso)

Sin mirar el documento, dibujá el **sistema completo** en una hoja: los 4 servicios, RabbitMQ en el medio, PostgreSQL colgando del `payment-service`, MercadoPago a la derecha del `adapter`, y el cliente arriba. Dibujá las flechas de **un** flujo completo a elección (el que más te costó) con sus exchanges y colas. Después abrí la Etapa 6 y **corregite**. Lo que te falte o equivoques es exactamente lo que conviene repasar primero cuando entres al código.

Fin de las etapas. Las respuestas a todas las preguntas de verificación están en `99-respuestas.md`.

Apéndice A — Fundamentos de Docker y Kubernetes desde cero

Objetivo del apéndice

Que entiendas **la maquinaria** que la Etapa 7 usaba sin abrir del todo. La Etapa 7 te enseñó a **operar** (mirar pods, abrir la UI, ver logs); este apéndice te enseña a **entender** cada pieza por dentro, partiendo de tu mundo de `.jar` en un Tomcat. Si la Etapa 7 era el manual del piloto, esto es el curso de cómo vuela el avión.

Todo lo de acá es **conceptual y genérico** (no depende del repo de GWP). Los `.yaml` son **ilustrativos**: el real puede variar en nombres y detalles, que confirmás en los manifiestos del proyecto.

Teoría

A.1 — Imagen, contenedor y registry (la base de todo)

Esta distinción es la que más confunde al principio, y sin ella nada cierra. Tres conceptos:

- **Imagen** — la *plantilla* inmutable: un paquete de solo-lectura con tu app + todo lo que necesita (binario/runtime, libs, config base). No corre; es como un molde.
- **Contenedor** — una *instancia en ejecución* de una imagen. De una imagen podés levantar 1, 5 o 50 contenedores idénticos. Es como pasar de la `class` al `new` que la instancia.
- **Registry** — el *repositorio* de imágenes (Docker Hub, un registry privado de la empresa). Hacés `push` para subir una imagen y los nodos hacen `pull` para bajarla antes de correrla.

Analogía directa con tu mundo: la **imagen** es el `.jar` empaquetado *junto con la JVM y el SO mínimo* (no solo tu código, sino el plato servido entero). El **contenedor** es ese `.jar` **ejecutándose** (`java -jar ...` corriendo). El **registry** es como tu **Nexus/Artifactory**, pero de imágenes en vez de `.jar`.

El Dockerfile y las capas. Una imagen se construye con un `Dockerfile`: una receta paso a paso. Cada instrucción genera una **capa** cacheada, así que reconstruir es rápido si solo cambió lo de arriba.

```
# Ejemplo ILUSTRATIVO – multi-stage con build nativo GraalVM
# Etapa 1: compilar a binario nativo (imagen pesada, con todo el toolchain)
FROM ghcr.io/graalvm/native-image AS build
COPY . /app
RUN cd /app && ./mvnw -Pnative native:compile

# Etapa 2: imagen final mínima – solo el binario, sin JVM ni Maven
FROM debian:stable-slim
COPY --from=build /app/target/app /app/app
ENTRYPOINT ["/app/app"]
```

El **multi-stage** es la clave del build nativo de GWP: la primera etapa (enorme: trae GraalVM, Maven, todo el toolchain) compila; la segunda **copia solo el binario** a una imagen mínima. Resultado: una imagen chica con un ejecutable que arranca en milisegundos. Por eso GWP puede correr varios pods livianos sin el costo de arranque/RAM de la JVM.

A.2 — Qué es `kubectl` y el "context"

`kubectl` es el **cliente de línea de comandos** que le habla al cluster. No "ejecuta" nada localmente: manda órdenes al **API server** de Kubernetes por HTTPS y te muestra la respuesta.

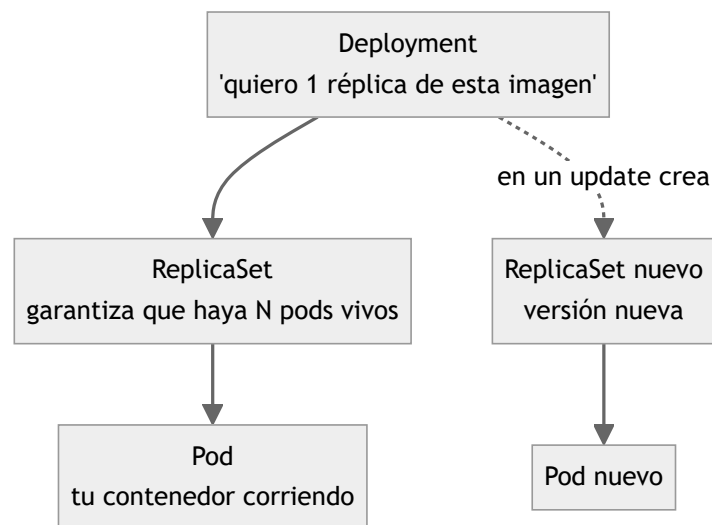
¿A qué cluster le habla? Eso lo define el **kubeconfig** (por defecto `~/.kube/config`), que guarda **contexts**: cada context es la terna (*cluster + usuario + namespace por defecto*). Minikube, al arrancar, te crea y selecciona su context automáticamente.

```
kubectl config current-context    # ¿a qué cluster le estoy hablando?
kubectl config get-contexts      # todos los contexts disponibles
kubectl config use-context minikube
```

Por qué te importa: si alguna vez un comando "no encuentra" tus pods, lo primero es chequear que `kubectl` esté apuntando al cluster correcto (el context). Es el equivalente a "¿me conecté a la base que creía?".

A.3 — La jerarquía: Deployment → ReplicaSet → Pod

En la Etapa 7 viste Pod y Deployment sueltos. En realidad hay una **cadena de mando**:



- **Pod** — la unidad mínima: 1 (o más) contenedores que comparten red y viven/mueren juntos. Es **efímero**: si se cae, ese pod no "revive" — nace **uno nuevo** con otro nombre/IP.
- **ReplicaSet** — el que **cuenta**: "tienen que haber N pods vivos". Si uno muere, levanta otro. Vos casi nunca lo tocás directo.
- **Deployment** — el que **gestiona versiones**: declarás la imagen y cuántas réplicas; cuando cambiás la imagen (deploy nuevo), el Deployment crea un **ReplicaSet nuevo** y va moviendo pods del viejo al nuevo (**rolling update**), pudiendo hacer **rollback** si algo sale mal.

Analogía: el **Deployment** es el supervisor que dice "quiero 1 operario haciendo esta tarea, versión 5 del manual". El **ReplicaSet** es el capataz que se asegura de que **siempre haya 1 operario vivo** (si se desmaya, mete otro). El **Pod** es el operario concreto, reemplazable.

Por qué "efímero" cambia tu forma de pensar: como los pods van y vienen con IP distinta, **nunca** le hablás a un pod por su IP. Para eso existe el Service (A.5). Y como un pod puede morir en cualquier momento, **su estado en disco se pierde** salvo que uses almacenamiento persistente (A.7) — por eso Postgres no puede ser un Pod común.

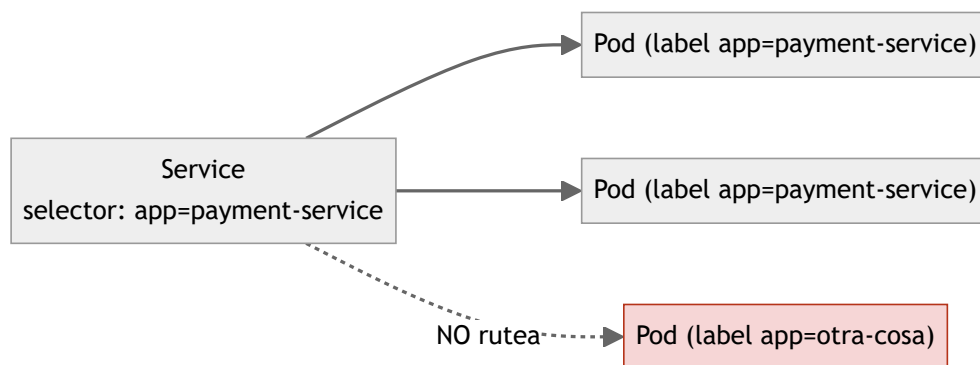
A.4 — Labels y selectors (el pegamento de Kubernetes)

Acá está **el mecanismo más importante** y el que la Etapa 7 no abría: **¿cómo sabe un Service a qué pods mandarles tráfico?** No por nombre ni por IP. Por **etiquetas (labels)**.

- Un **label** es un par clave-valor que le pegás a un objeto: `app: payment-service`, `tier: backend`.
- Un **selector** es una *consulta* por labels: "todos los pods con `app=payment-service`".

Casi todas las conexiones de K8s funcionan así: un Deployment sabe qué pods son "suyos" por selector; un Service sabe a qué pods rutear por selector.

```
# Ejemplo ILUSTRATIVO – el Service encuentra a sus pods por label
apiVersion: v1
kind: Service
metadata:
  name: payment-service
spec:
  selector:
    app: payment-service    # <-- rutea a TODO pod con este label
  ports:
    - port: 8080
```



Analogía: el selector es un `WHERE label = 'payment-service'`. El Service no tiene una lista fija de pods; **pregunta dinámicamente** "¿quiénes matchean?" y reparte entre ellos. Por eso cuando un pod muere y nace otro (con otra IP) el Service **se acomoda solo**: el nuevo pod tiene el mismo label, así que entra a la rotación sin que nadie toque nada.

Esto es lo que hace que el sistema sea *self-healing*: las piezas se encuentran por **identidad lógica (labels)**, no por dirección física.

A.5 — Service y DNS interno (cómo se encuentran los servicios)

Un **Service** te da dos cosas estables sobre un grupo de pods efímeros: una **IP virtual fija** y, sobre todo, un **nombre DNS**. Kubernetes corre un DNS interno: cada Service es resoluble por su nombre **dentro del cluster**.

Esto responde la pregunta que quedaba colgada: *¿cómo sabe el payment-service dónde está PostgreSQL o RabbitMQ?* **Por nombre**. En su configuración, la URL de la base no es una IP: es algo como `postgres-service:5432`, y el de RabbitMQ `rabbitmq-service:5672`. El DNS del cluster traduce ese nombre a la IP del Service, que a su vez reparte a los pods.

```
# Nombre DNS completo de un Service (forma larga)
<service>.<namespace>.svc.cluster.local
ej.: postgres-service.dev.svc.cluster.local
# Dentro del mismo namespace alcanza con el nombre corto: "postgres-service"
```

Repaso de los tipos (ya en la Etapa 7), ahora con el "por qué":

- **ClusterIP** (default) — IP/nombre **solo interno**. Es como hablan entre sí los servicios de GWP. No accesible desde afuera.
- **NodePort** — abre un puerto en el nodo hacia afuera (`:30000` del api-gateway). Es la "puerta de calle".
- **Headless** (`clusterIP: None`) — **no** da una IP única; en cambio expone **un DNS por cada pod** (`rabbitmq-0.<service>...`). Se usa con StatefulSets, donde te importa hablarle a *un pod específico* por su identidad estable, no a "cualquiera del montón".

Analogía: el Service es un **número de interno** de la empresa (siempre el mismo) que el conmutador (DNS + balanceo) deriva al empleado disponible. Vos llamás al interno "Contaduría", no al celular personal de Juan, que cambia.

A.6 — ConfigMap y Secret: cómo entran de verdad al contenedor

La Etapa 7 dijo *qué* son. Falta el *cómo*: la config no se "lee mágicamente", se **inyecta** al pod de una de dos formas:

1. **Como variables de entorno** — cada clave del ConfigMap/Secret se vuelve un `ENV` que tu app lee (en Spring Boot, eso alimenta tus `@Value / application.yml` vía variables de entorno).
2. **Como archivos en un volumen montado** — el ConfigMap/Secret aparece como archivos dentro del contenedor (ej. `/etc/secrets/webhook-secret`).

```
# Ejemplo ILUSTRATIVO — inyectar config como env vars
envFrom:
- configMapRef: { name: payment-config } # URLs, flags
- secretRef:   { name: payment-secrets } # tokens, passwords
```

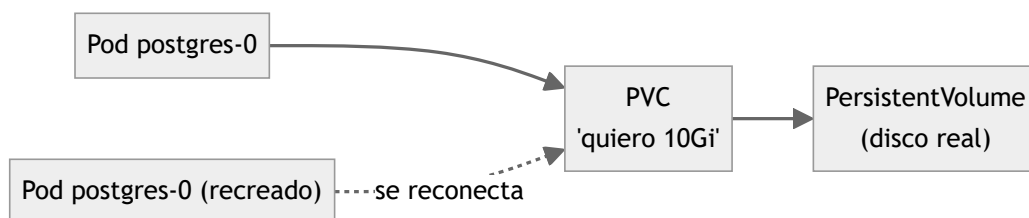
El malentendido peligroso que tenés que sacarte de encima: un **Secret NO está encriptado**. Su contenido está en **base64**, que es solo una *codificación* (cualquiera lo decodifica en dos segundos). La diferencia real con un ConfigMap es de **control de acceso y manejo** (RBAC, no se loguea, se puede cifrar en reposo si el cluster lo configura), no de "está escondido". Por eso los **tokens de MercadoPago y los webhook secrets por cliente** van en Secrets: para tratarlos con cuidado, no porque base64 los proteja.

```
# Esto "desencripta" un Secret... porque nunca estuvo encriptado:
echo "dG9rZW4tc2VjcmV0bw==" | base64 -d
```

A.7 — Persistencia real: PV, PVC y por qué Postgres es un StatefulSet

Recordá A.3: un pod es **efímero**, y lo que escribe en su disco **se borra** cuando muere. Para una base de datos eso es inaceptable. La solución de Kubernetes son **volúmenes persistentes**, con dos piezas:

- **PersistentVolume (PV)** — un pedazo de almacenamiento *real* del cluster (un disco). Existe independientemente de los pods.
- **PersistentVolumeClaim (PVC)** — un *pedido* de almacenamiento que hace un pod: "necesito 10Gi". Kubernetes lo **vincula** a un PV que lo satisfaga. El pod usa el PVC; si el pod muere y nace otro, el nuevo **se reconecta al mismo PVC** y recupera los datos.



Acá cierra lo del StatefulSet (de la Etapa 7): un **StatefulSet** existe justamente para cargas con estado. A diferencia de un Deployment, le da a cada pod:

1. **Identidad estable** — nombres fijos y ordenados (`postgres-0` , `rabbitmq-0`), no aleatorios.
2. **Su propio PVC persistente** — atado a ese pod. Si `postgres-0` se recrea, vuelve a engancharse a su mismo disco. Los datos sobreviven.

Por eso Postgres y RabbitMQ **no pueden** ser Deployments comunes: perderían identidad y datos en cada reinicio. Los microservicios de GWP sí son Deployments porque son **stateless** (su "estado" vive en Postgres o en las colas, no en su disco).

Analogía: un Deployment es personal *temporario* intercambiable (cualquiera hace la tarea, no tiene escritorio fijo). Un StatefulSet es personal *con escritorio asignado y archivero propio*: `postgres-0` siempre vuelve a SU archivero (PVC), aunque la persona cambie.

A.8 — Minikube por dentro y el error que Sí o Sí vas a tener

Minikube no es Kubernetes "instalado en tu Windows": es un **cluster de 1 nodo corriendo dentro de una VM o un contenedor** en tu máquina. Tiene **su propio Docker/almacén de imágenes, separado del Docker de tu laptop**. Y ahí está la trampa número uno del que arranca:

`ImagePullBackOff` / `ErrImagePull` : construiste la imagen en *tu* Docker, hacés `kubectl apply` , y el pod queda en `ImagePullBackOff` . ¿Por qué? Porque Minikube busca la imagen en **su** almacén (o en un registry remoto) y **ahí no está**: la imagen vive en el Docker de tu host, que Minikube no ve.

Las salidas habituales (confirmá cuál usa GWP en su documentación de arranque):

```
# Opción A: cargar la imagen de tu host al almacén de Minikube
minikube image load mi-imagen:tag

# Opción B: construir directamente DENTRO del Docker de Minikube
# (apunta tu CLI de Docker al daemon de Minikube en esta shell)
eval $(minikube docker-env)      # en PowerShell: minikube docker-env | Invoke-Expression
docker build -t mi-imagen:tag .

# Y que el Deployment NO intente bajarla de internet:
#   imagePullPolicy: IfNotPresent (o Never para imágenes 100% locales)
```

Otros dos comandos de Minikube que te van a servir:

```
minikube start                # levanta el cluster
minikube service <svc> -n dev --url # te da una URL accesible a un NodePort
minikube dashboard            # la UI visual (ya la viste en Etapa 7)
```

Por qué te lo marco fuerte: este error desconcierta a *todos* los que empiezan ("¡pero si la imagen existe, la acabo de construir!"). Entender que Minikube tiene su propio almacén te ahorra una tarde de frustración.

A.9 — Liveness y readiness probes (por qué un pod "reinicia solo")

Kubernetes no sabe si tu app *está sana* solo porque el proceso vive. Para eso le das **probes**:

- **Liveness probe** — "¿sigue vivo y sano?". Si falla repetidamente, K8s **mata y recrea** el pod. Un servicio colgado (deadlock) se recupera solo.
- **Readiness probe** — "¿está listo para recibir tráfico?". Si falla, K8s lo **saca del Service** (no le rutea) hasta que se recupere, sin matarlo. Útil en el arranque o cuando un dependiente (RabbitMQ) todavía no está.

Conexión con la Etapa 8: cuando ves un pod en `CrashLoopBackOff` , muchas veces es una **liveness probe** que falla (la app no levanta bien, o no conecta a RabbitMQ/Postgres) y K8s lo reinicia una y otra vez. `kubectl describe pod` te muestra exactamente qué probe falló.

En nuestro sistema (cómo se arma todo junto)

Juntando las piezas, el "arranque" mental de GWP en Minikube es:

1. Se **construyen las imágenes** nativas (multi-stage GraalVM) y se ponen al alcance de Minikube (A.1, A.8).

2. Se **aplican los manifiestos** (`kubectl apply`): se crean Deployments (4 microservicios), StatefulSets (`postgres-0` , `rabbitmq-0`), Services, ConfigMaps y Secrets, todo en el namespace `dev` .
3. Cada **Deployment** levanta su ReplicaSet → su Pod (A.3). Los **StatefulSets** levantan sus pods con identidad estable y enganchan sus **PVC** (A.7).
4. Los pods se **encuentran entre sí por Service + DNS** (`rabbitmq-service` , `postgres-service`) usando **labels/selectors** (A.4, A.5).
5. La config y los secretos (tokens MercadoPago, webhook secrets) se **inyectan** desde ConfigMap/Secret (A.6).
6. El tráfico externo entra: Nginx → NodePort del api-gateway (`:30000`) → pod (`:8080`).

Qué buscar en el código / repo (para la semana)

- En cada manifiesto, los **labels** (en `metadata.labels`) y los **selector** (en Deployments y Services). Verificá que el selector de un Service matchea los labels de los pods del Deployment correspondiente. **Ese match es la conexión.**
- En los Services, el campo **type** (`ClusterIP` / `NodePort` / `clusterIP: None` para headless) y cómo la **config de los microservicios** referencia a `rabbitmq-service` / `postgres-service` **por nombre** (DNS interno).
- En los StatefulSets, la sección **volumeClaimTemplates** (los PVC por pod) y dónde se monta el volumen.
- En los Deployments, **imagePullPolicy** y el nombre/tag de la **imagen** (para entender el tema Minikube de A.8).
- Las **probes** (`livenessProbe` / `readinessProbe`) en los Deployments: qué endpoint o chequeo usan.
- El o los **Dockerfile** (multi-stage GraalVM) — confirmá las dos etapas de A.1.

Preguntas para verificar que entendiste

1. Diferenciá **imagen**, **contenedor** y **registry** con la analogía del `.jar` . ¿Qué rol cumple el multi-stage en el build nativo?
2. ¿Cómo sabe un **Service** a qué pods mandar tráfico, si las IP de los pods cambian todo el tiempo? Nombrá el mecanismo.
3. ¿Cómo encuentra el payment-service a PostgreSQL y a RabbitMQ sin conocer sus IP?
4. Verdadero o falso: "los datos de un Secret están encriptados". Justificá y decí cuál es la diferencia real con un ConfigMap.
5. Explicá por qué Postgres es un **StatefulSet** y no un Deployment, usando los conceptos de **pod efímero** y **PVC**.
6. Construiste la imagen en tu Docker, hacés `kubectl apply` y el pod queda en `ImagePullBackOff` . ¿Por qué pasa en Minikube y cómo lo resolvés?

Mini-ejercicio

Tomá el flujo de "una solicitud de pago entra al sistema" y, **en una hoja, anotá cada pieza de infraestructura que el tráfico/los datos atraviesan**, en orden: Nginx → NodePort → Service del gateway → pod del gateway → (RabbitMQ por DNS) → ... → payment-service → (Postgres por DNS) → PVC → disco. Al lado de cada salto, escribí **qué concepto de este apéndice explica ese salto** (label/selector, DNS interno, PVC, etc.). El objetivo: que la palabra "infraestructura" deje de ser una caja negra y se vuelva una cadena de piezas que podés nombrar.

Las respuestas están en `99-respuestas.md` (sección Apéndice A).

Siguiente: `99-respuestas.md`

Respuestas a las preguntas de verificación

Leé estas respuestas **después** de intentar las preguntas vos. Son guías, no dogma: si tu redacción dice lo mismo con otras palabras, está perfecto.

Eta 0 — Por qué eventos

1. **Disponibilidad atada a MercadoPago.** Porque en el monolito la llamada a MercadoPago es **sincrónica y bloqueante** dentro del request: si MercadoPago tarda o está caído, tu endpoint tarda o falla, aunque la solicitud era válida. GWP rompe la atadura porque el gateway **publica un evento y responde 202 al instante**; hablar con MercadoPago pasa a ser tarea asíncrona del adapter, que reintenta cuando el proveedor vuelve. La disponibilidad del gateway ya no depende de la de MercadoPago.
2. **202 vs 200.** 200 OK dice "terminé, acá está el resultado". 202 Accepted dice "**recibí tu pedido y me hago cargo**, pero todavía no terminé". Es honesto con la naturaleza asíncrona: el resultado (el QR, la confirmación) llega después, por el webhook del cliente.
3. **Los cuatro problemas:** acoplamiento temporal, escalado en bloque, fallos en cascada y asincronía artesanal (a mano con `BlockingQueue` / `Executor`, sin durabilidad).
4. **El costo principal** es la **consistencia eventual** (más debugging distribuido y complejidad operativa). Vale la pena en pagos porque **no perder un mensaje y no caerse cuando un proveedor se cae** importan más que la simplicidad de un monolito; el sistema converge a un estado correcto sin perder información.
5. **`BlockingQueue` ≠ `RabbitMQ`** porque la `BlockingQueue` vive en el **heap de tu proceso**: si el proceso muere, los ítems se pierden, no hay ack/nack ni reintento garantizado, no es compartible entre servicios ni durable. `RabbitMQ` es externo, durable, con entrega garantizada (at-least-once) y DLQ.

Eta 1 — Vocabulario de mensajería

1. **Productor → exchange (no a la cola)** para **desacoplar** al productor de los consumidores: no necesita saber cuántas colas hay ni quién consume. Agregás/quitás consumidores cambiando bindings, sin tocar al productor. La indirección es lo que da la flexibilidad de ruteo.
2. **fanout con rk ""**: en fanout la routing key **se ignora** (por eso va vacía); el mensaje se copia a todas las colas atadas. Para sumar un consumidor nuevo, **atás otra cola** al exchange y listo — el gateway ni se entera.
3. **ack vs nack**: ack = "procesado OK, el broker borra el mensaje"; nack = "falló, se re-encola o va a DLQ según config". Si el consumidor **se cae antes de ack o nack**, el broker detecta la conexión muerta y **redan** el mensaje a otro consumidor (no se pierde).
4. **at-least-once ⇒ idempotencia** porque, como un mensaje puede entregarse **más de una vez** (caída antes del ack, redelivery, reintentos, Outbox que republica), procesarlo dos veces no debe duplicar efectos. La única forma segura es que

el procesamiento sea idempotente.

5. **DLQ** aísla los mensajes que fallan repetidamente para que **un poison message no gire infinito bloqueando la cola** ni se pierda. Quedan apartados y visibles para inspección humana.

Etapa 2 — RabbitMQ en este proyecto

1. **Beans de topología:** al arrancar, un `RabbitAdmin` **crea en el broker** los exchanges/colas/bindings declarados (si no existen). Conviene en código porque la topología queda **versionada, reproducible y revisable**, no dependiente de clicks manuales que nadie recuerda.
2. **Ack con `auto`:** lo hace **Spring** (el listener container) automáticamente: si el método retorna sin excepción → **ack**; si lanza excepción → **nack** (y según config, re-encola o DLQ).
3. **JSON, no objetos Java**, porque los servicios son **procesos/contenedores separados** que no comparten memoria; el contrato común es el JSON sobre el cable. Implica que cambiar un campo de un evento es un **cambio de contrato**: todos los consumidores tienen que entender la nueva forma (versionado de eventos).
4. **5.000 ready, 0 unacked:** no hay **consumidor vivo** tomando mensajes (o todos están desconectados). Los mensajes entran y se acumulan porque nadie los consume.
5. **max-concurrency** define cuántos consumidores concurrentes puede levantar un listener cuando hay backlog. Se relaciona con el escalado de la Etapa 0: permite **procesar más rápido la cola saturada** sumando consumidores, sin escalar el resto.

Etapa 3 — Hexagonal + Clean + CQRS

1. **Dominio sin Spring/JPA/RabbitMQ** para que la **lógica de negocio sea pura y testeable sin infraestructura** (le pasás mocks de los puertos) y para poder **cambiar base/broker/proveedor sin tocar el negocio**. Ganás testeo rápido y resistencia al cambio.
2. **Puerto de entrada:** cómo el mundo externo dispara el negocio — en el payment-service, los `@RabbitListener` que reciben eventos. **Puerto de salida:** lo que el negocio necesita del mundo — el repositorio PostgreSQL y el publicador de eventos. (Los adapters los implementan.)
3. **Command / Event / Query:** command = "hacé esto" (puede rechazarse, imperativo); event = "esto ya pasó" (hecho inmutable, pasado); query = "decime esto" (sin efectos). `OnPaymentRequestReceivedEvent` es un **evento** porque describe un **hecho consumado** (se recibió la solicitud), no una orden que alguien pueda rechazar.
4. **Dependencias:** `adapters → application → domain`, **nunca al revés**. Así el dominio no conoce detalles externos y se mantiene puro y estable; lo volátil (frameworks, proveedores) queda en los bordes.
5. **Commands record inmutables** porque se **serializan y pasan entre hilos/servicios**: la inmutabilidad los hace seguros para concurrencia (no hay estado mutable compartido) y predecibles (una vez creado el command, su contenido no cambia).

Etapa 4 — Recorrido de los 4 microservicios

1. **Gateway sin BD, payment-service con BD:** el gateway solo **traduce HTTP→evento** y no es dueño de ningún estado, así que no necesita base (es stateless y escalable trivialmente). El payment-service es el **Aggregate Root**: dueño de la verdad del estado de cada pago, por eso necesita persistencia (PostgreSQL).
2. **Cómo le llegan cosas al payment-service:** (a) por **mensajes** de las 9 colas que consume, y (b) por sus **schedulers internos** (`ExpiredOrdersScheduler` 5s, `RefundInProgressScheduler` 60s) — el tiempo como disparador.
3. **El adapter encapsula** lo del proveedor: por ejemplo, el **formato de la API REST de MercadoPago** (endpoints, payloads), la **autenticación con token/secret por cliente**, la **validación de firma HMAC**, y los **reintentos/DLQ**. El resto del sistema no debería conocer nada de eso.
4. **Strategy de los 12 extractores:** evita un `switch / if-else` gigante por proveedor. Cada proveedor tiene **su implementación** de la interfaz "extractor de metadatos", elegida en runtime. Agregar un proveedor = **una clase nueva**, sin tocar las otras ni arriesgar regresiones.
5. `on_notification_attempt_completed` es **auditoría**: registra el resultado de cada intento de notificación (exitoso o no). No hace falta para el "happy path", pero da trazabilidad, métricas y soporte (¿le avisamos al cliente?, ¿cuántos reintentos?, ¿falló?).

Etapa 5 — Patrones

1. **Dual write:** hay que escribir a **dos sistemas** (PostgreSQL + RabbitMQ) que no comparten transacción. Un `@Transactional` solo cubre la base; si la app cae **entre el commit y el publish**, guardaste el estado pero **perdiste el evento** (o publicaste un evento sobre algo que no se commiteó). No hay atomicidad entre los dos sistemas.
2. **Outbox = single write:** el evento se inserta en `outbox_events` **en la misma transacción** que la entidad → una sola escritura atómica. La publicación real se hace después (publish inmediato vía `ApplicationEvent`); si falla, el **scheduler de respaldo (3s, máx 3 intentos)** publica los pendientes desde la tabla. El evento **nunca se pierde**.
3. **at-least-once ⇒ idempotencia:** porque el mismo mensaje puede llegar varias veces. Idempotente: "confirmar la orden #123" (queda confirmada una vez, lleguen 1 o 3 avisos). No idempotente: "sumar \$100 al saldo" (3 entregas = +\$300). En pagos, lo segundo sería cobrar de más.
4. **Consistencia eventual:** el sistema no está coherente en *todo instante*, pero **converge** al rato. Es aceptable porque a cambio ganas desacople y disponibilidad sin transacciones distribuidas. Se nota, por ejemplo, en la ventana entre que el payment-service **confirma** el pago y el notification-service **avisa** al cliente.
5. **A la DLQ** llega un mensaje que **falla repetidamente** (se agotan reintentos) o se rechaza sin re-encolar. Una DLQ que **crece es alarma** porque significa que hay mensajes que ningún reintento pudo procesar: bug, dato inválido o dependiente caído — requiere intervención humana.

Etapa 6 — Flujos end-to-end

1. **Flujo 1, respuesta al cliente:** el cliente recibe **202 en el paso 1**, apenas el gateway publica el evento (mucho antes de que el QR exista). Se entera de que el QR está listo **por su webhook**, cuando el notification-service le manda el POST firmado.

2. **GET /v1/orders/{id} en vez de confiar en el webhook:** porque el webhook es solo un **aviso** (puede ser falsificado, llegar duplicado o estar incompleto). El adapter **valida la firma HMAC** y luego **consulta el estado real** a MercadoPago para actuar sobre el dato autoritativo, no sobre el aviso.
3. **Flujo 3 lo dispara un scheduler** del payment-service (`ExpiredOrdersScheduler 5s` / `RefundInProgressScheduler 60s`): el disparador es **el tiempo**, no un request HTTP. Eso lo distingue de los demás, que arrancan con una acción del cliente o de MercadoPago.
4. **RefundInProgressScheduler (60s):** MercadoPago puede procesar reembolsos de forma **asíncrona**, dejándolos "en progreso". El scheduler hace un **loop de consulta** periódica del estado hasta que el reembolso se cierra, para que el sistema no quede esperando indefinidamente un aviso que quizá no llega.
5. **Los 5 casilleros de cualquier salto:** (1) **evento** emitido, (2) **exchange**, (3) **routing key**, (4) **cola** destino, (5) **caso de uso/consumidor** que lo procesa.

Etapas 7 — Infraestructura

1. **StatefulSets** porque RabbitMQ y PostgreSQL tienen **estado e identidad estable**: necesitan nombre fijo (`rabbitmq-0` , `postgres-0`) y **almacenamiento persistente** que sobreviva reinicios. Un Deployment común trata a los pods como intercambiables y efímeros, lo que rompería una base o un broker.
2. **NodePort vs ClusterIP:** NodePort **expone un puerto hacia afuera** del cluster (accesible desde el nodo); ClusterIP es **solo interno**. El api-gateway usa NodePort (`:30000` → `:8080`) porque es el **único punto de entrada HTTP** y tiene que ser alcanzable desde afuera (vía Nginx).
3. **port-forward 15672** abre un túnel desde tu `localhost:15672` a la UI de Management de RabbitMQ. Una vez conectado, mirás **exchanges (tipos/bindings)** y **colas (profundidad, ready/unacked)** para ver el sistema en vivo y diagnosticar.
4. **Cola del payment-service crece, la del adapter en 0:** el **payment-service no está consumiendo** (caído, en `CrashLoopBackOff` , o lanzando excepciones en loop). Confirmás con `kubectl get pods -n dev` y `kubectl logs -n dev <pod-payment-service>` .
5. **Tokens y webhook secrets por cliente** viven en **Secrets** de Kubernetes (no en ConfigMaps) porque son **datos sensibles**; los Secrets están pensados para credenciales (control de acceso, no se exponen como config plana).

Etapas 8 — Consolidar

1. **Agregar "PagoYa":** el que **más cambia** es un **nuevo adapter** (`pagoya-adapter`) que encapsula todo lo del proveedor. El **api-gateway** casi no cambia (solo un **binding** nuevo por la routing key del proveedor en el exchange topic). El **payment-service** casi no cambia (razona en eventos de dominio genéricos). El **notification-service** suma **una Strategy** (un extractor) si la metadata difiere.
2. **El webhook debe ser topic** porque la routing key es el `providerCode` : con topic podés **agregar un binding** para rutear las notificaciones del proveedor nuevo a su propia cola **sin tocar el gateway**. Con `fanout` todas las colas recibirían todo (no podrías separar por proveedor) y con `direct` igual sirve por key exacta, pero `topic` da la flexibilidad de patrones por si mañana querés agrupar proveedores.

3. **Investigar la DLQ:** (1) abrir un mensaje en la UI (payload + headers `x-death` con el motivo), (2) cruzar con los **logs del consumidor** en la ventana del fallo, (3) decidir bug (fix + reproceso) vs dato inválido (descartar).
4. **ready alto + unacked 0:** no hay consumidor vivo tomando mensajes. **unacked alto y estable:** el consumidor **tomó** mensajes pero **no ackea** — está colgado/lento procesando (probablemente esperando a un tercero o trabado).
5. **Evento que no llega — el camino:** recorrer **productor** → **exchange** → **binding** → **cola** → **consumidor**, preguntando en cada salto "¿llegó hasta acá?". Chequear: ¿publicó? (logs / `outbox_events`), ¿el binding/routing key es correcto? (UI Exchanges), ¿la cola tiene consumidor? (Síntoma 2), ¿terminó en la DLQ? (Síntoma 1). El primer "no" marca el problema.

Apéndice A — Fundamentos de Docker y Kubernetes

1. **Imagen / contenedor / registry:** la **imagen** es la plantilla inmutable (como el `.jar` empaquetado con la JVM y el SO mínimo: el plato servido entero); el **contenedor** es esa imagen **en ejecución** (el `.jar` corriendo); el **registry** es el repositorio de imágenes (tu Nexus/Artifactory, pero de imágenes). El **multi-stage** separa el build del runtime: la primera etapa (pesada: GraalVM + toolchain) **compila el binario nativo**, la segunda **copia solo el binario** a una imagen mínima → imagen chica que arranca en milisegundos.
2. **Cómo el Service encuentra a sus pods:** por **labels y selectors**, no por IP. El Service tiene un `selector` (ej. `app=payment-service`) y rutea dinámicamente a **todo pod que matchee ese label**. Cuando un pod muere y nace otro con el mismo label, entra solo a la rotación: por eso el sistema es *self-healing* y nadie toca IPs.
3. **Cómo encuentra el payment-service a Postgres/RabbitMQ:** por el **DNS interno** del cluster. En su config no hay IPs sino **nombres de Service** (`postgres-service:5432`, `rabbitmq-service:5672`); el DNS resuelve ese nombre a la IP del Service, que reparte a los pods.
4. **Falso.** Un Secret está en **base64**, que es solo *codificación* (se decodifica trivialmente, no es cifrado). La diferencia real con un ConfigMap es de **manejo y control de acceso** (RBAC, no se loguea, se puede cifrar en reposo si el cluster lo habilita), no de ocultamiento. Por eso los tokens/webhook secrets van en Secret: para tratarlos con cuidado, no porque base64 los proteja.
5. **Postgres como StatefulSet:** un Pod es **efímero** — si muere, nace otro con otra IP y **su disco se pierde**. Una base no puede perder datos en cada reinicio. El **StatefulSet** le da al pod **identidad estable** (`postgres-0`) y un **PVC** propio atado a él: si `postgres-0` se recrea, **se reconecta al mismo PVC/disco** y recupera los datos. Un Deployment trata a los pods como intercambiables y sin disco propio, así que no sirve para estado.
6. **ImagePullBackOff en Minikube:** porque Minikube corre en su **propia VM/contenedor con su propio almacén de imágenes**, separado del Docker de tu host. Construiste la imagen en *tu* Docker; Minikube la busca en *el suyo* (o en un registry remoto) y no la encuentra. Se resuelve **cargándola** (`minikube image load <img>`), **construyéndola dentro del Docker de Minikube** (`minikube docker-env + docker build`), y poniendo `imagePullPolicy: IfNotPresent / Never` para que no intente bajarla de internet.